

Grundlagen von Datenbanken

Datenbanken

Lernzettel

Kompakte Zusammenfassung der Vorlesungsinhalte für die Prüfungsvorbereitung.

Autor Levin Rüßmann
Matrikelnummer 838791
Studiengang Informatik B.Sc.
Semester Sommersemester 2026
Datum 5. Juli 2026

Inhaltsverzeichnis

1	Datenbanken — Grundlagen	3
1.1	Datenbank – Definition	3
1.2	Datenbank-Management-System – DBMS	3
1.2.1	Anforderungen an ein DBMS	3
1.2.2	Aufbau eines DBMS	4
2	Datenbankmodelle	5
2.1	Relationales Datenbankmodell	5
2.2	Hierarchisches Datenbankmodell	6
2.3	Netzwerkmodell	7
2.4	Objektorientiertes Datenbankmodell	8
3	Datenverteilung	9
3.1	Gründe für die Datenverteilung	9
4	Datenbanken in verteilten Systemen	10
4.1	CAP-Theorem	10
4.2	Zentrale Datenbank	11
4.3	Replikation	11
4.3.1	Arten	11
4.3.2	Modus	12
4.4	Sharding	12
4.5	Caching	13
4.6	Map/Reduce	13
5	Managementunterstützungssysteme	14
5.1	Data Warehouse	14
5.2	Data Mart	15
5.3	OLTP und OLAP	16
5.3.1	Online Transaction Processing – OLTP	16
5.3.2	Online Analytical Processing – OLAP	16
5.3.3	Dimensionen und Fakten	16
5.3.4	Star-Schema	17
5.4	Exkurs: CQRS	19
5.5	Data Mining	19
6	Datenbank-Architektur	21
6.1	ANSI-3-Ebenen-Modell	21
6.1.1	Interne Ebene	21
6.1.2	Konzeptionelle Ebene	21
6.1.3	Externe Ebene	22
6.2	Datenbank-Entwurfsphasen	23
6.2.1	Planungsphase	23
6.2.2	Definitionsphase	24
6.2.3	Entwurfsphase	24
6.2.4	Implementierungsphase	24
6.2.5	Abnahme- und Einführungsphase	24
6.2.6	Wartungs- und Pflegephase	24
7	NoSQL	25

7.1	Modelle	25
7.1.1	Key-Value	25
7.1.2	Document Store	25
7.1.3	Column Store	25
7.1.4	Graphdatenbank	25
8	Datenmodellierung	26
8.1	Stufen der Datenmodellierung	26
8.2	Entity-Relationship-Modell	26
8.2.1	Chen-Notation	26
8.3	Vom ERM zum RDM	27
8.4	UML-Klassendiagramm	28
9	Normalisierung	31
9.1	Anomalien und Redundanz	31
9.2	Funktionale Abhängigkeit	31
9.3	Erste Normalform (1NF)	31
9.4	Zweite Normalform (2NF)	32
9.5	Dritte Normalform (3NF)	32
9.6	Zusammenfassung	33
10	SQL	34
10.1	DDL – Data Definition Language	34
10.1.1	Datenbank und Tabelle anlegen	34
10.1.2	Datentypen	35
10.1.3	Spaltenattribute und Constraints	35
10.1.4	Struktur ändern mit ALTER TABLE	35
10.1.5	Referenzielle Integrität	36
10.1.6	Löschen: DROP, TRUNCATE, DELETE	36
10.2	DML – Data Manipulation Language	36
10.2.1	INSERT	37
10.2.2	UPDATE, REPLACE und DELETE	37
10.3	DQL – Data Query Language	37
10.3.1	Aufbau eines SELECT-Statements	37
10.3.2	WHERE – Zeilen filtern	38
10.3.3	Funktionen	39
10.3.4	GROUP BY und HAVING	39
10.3.5	ORDER BY und LIMIT	40
10.3.6	Joins	40
10.3.7	Unterabfragen (Subselects)	41
10.3.8	Views (Sichten)	41
10.4	DCL – Data Control Language	42
10.5	TCL – Transaktionen	42
10.5.1	Locking	43
10.5.2	Isolationsebenen	43
10.5.3	Events	43
10.6	Trigger, Funktionen und Prozeduren	43
10.6.1	Trigger	43
10.6.2	Funktionen	43
10.6.3	Prozeduren (Stored Procedures)	44
11	Glossar	45

1 Datenbanken — Grundlagen

1.1 Datenbank – Definition

Eine Datenbank speichert eine Vielzahl von Daten nach einem bestimmten Datenbankmodell. Im Gegensatz zu einem einfachen Verzeichnis werden die Daten strukturiert und redundanzfrei gespeichert. Außerdem ermöglicht eine Datenbank die dauerhafte (persistente) Speicherung von Daten.

1.2 Datenbank-Management-System – DBMS

Eine Datenbank wird immer durch ein Datenbank-Management-System (DBMS) verwaltet. Beispiele dafür sind MySQL, PostgreSQL oder Microsoft SQL Server. Das DBMS ist die Schnittstelle zur Datenbank – egal ob per CLI oder über eine GUI. Außerdem ermöglicht ein DBMS den Mehrnutzerbetrieb.

Vorteile

- Redundanzfreiheit – Daten werden nur einmal gespeichert
- Flexibilität – Struktur lässt sich nachträglich anpassen
- Physische Datenunabhängigkeit – Anwendung ist von der Speicherform getrennt
- Mehrnutzerbetrieb – mehrere Nutzer können gleichzeitig zugreifen
- Datenintegrität – Konsistenz wird durch das DBMS sichergestellt
- Recovery – Wiederherstellung nach einem Fehler oder Absturz
- Zugriffskontrolle – Rechte können nutzerspezifisch vergeben werden

Nachteile

- Ressourcenintensiv – benötigt mehr Rechenleistung und Speicher
- Komplexität – Einrichtung und Administration erfordern Fachwissen
- Kosten – Anschaffung, Lizenzen und Wartung verursachen laufende Ausgaben
- Schulungsaufwand – Mitarbeiter müssen im Umgang mit dem DBMS geschult werden
- Single Point of Failure – fällt das DBMS aus, sind alle Daten nicht verfügbar

1.2.1 Anforderungen an ein DBMS

1. **Redundanzabbau:** Durch Normalisierung verwaltet das DBMS die Daten so, dass sie möglichst wenig redundant gespeichert werden.
2. **Datenintegrität:** Die gespeicherten Daten müssen korrekt und zuverlässig sein.
3. **Effizienz und Performance:** Daten sollen schnell abgefragt werden können.
4. **Benutzerfreundlichkeit:** Tabellen sollten verständlich benannt sein, damit Daten leicht zu finden sind und in sinnvollen Datentypen vorliegen.
5. **Mehrfachzugriff:** Mehrere Nutzer sollen gleichzeitig auf die Datenbank zugreifen können.
6. **Datenschutz:** Daten werden durch Nutzerrollen und Anonymisierung geschützt.
7. **Datensicherheit:** Daten werden durch Backups gesichert und geschützt.
8. **Datenunabhängigkeit:** Grad, in dem Nutzer und Anwendung unabhängig von der

konkreten Speicherform auf das DBMS zugreifen können.

1.2.2 Aufbau eines DBMS

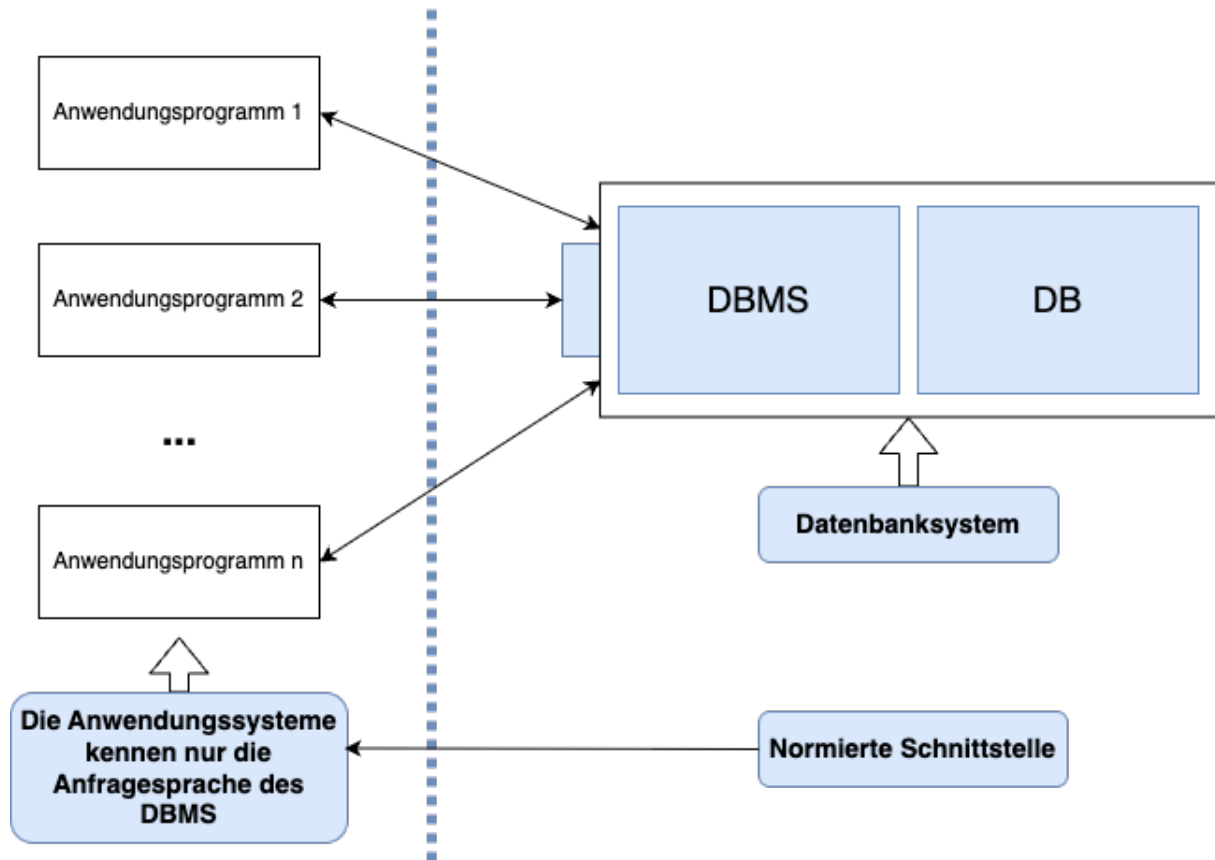


Abbildung 1. Aufbau eines DBMS

Auch bei dem Aufbau kann man Elemente wie den Mehrnutzerzugriff erkennen. Auch die Datenunabhängigkeit wird gezeigt, da die Anwendung nur die Anfragesprache kennt, aber nicht die Datenbank. Die Anfragesprache ist meist ein SQL-Dialekt.

2 Datenbankmodelle

Datenbankmodelle entscheiden, wie Daten gespeichert und wie sie abgerufen werden. Jedes der folgenden Datenbankmodelle hat einen eigenen sinnvollen Use Case. Keins der Modelle ist schlecht, trotzdem hat sich heutzutage das relationale Datenbankmodell durchgesetzt.

2.1 Relationales Datenbankmodell

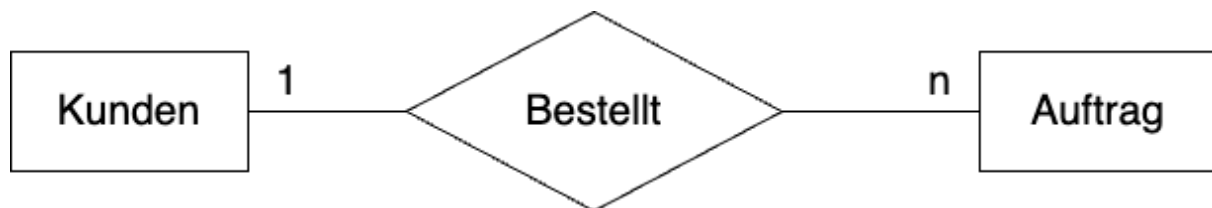


Abbildung 2. Beispiel: relationales Datenbankmodell

Ein relationales Datenbankmodell besteht aus *Relationen*, also Tabellen. Jede Zeile ist ein *Tupel*. Ein Tupel ist eine Liste mit fester Länge, die nicht mehr geändert werden kann. Im Fall des relationalen Datenbankmodells bestehen die Tupel aus einer Reihe von Attributen. Das Schema einer Relation legt dabei die Anzahl und den Typ der Attribute fest. Ein Datensatz in der Tabelle wird `Record` genannt.

Vorteile

- Große Verbreitung
- Einfach und flexibel

Nachteile

- Aufwendige Segmentierung
- Künstliche Schlüsselattribute wie IDs
- Anspruchsvolle Komposition

2.2 Hierarchisches Datenbankmodell

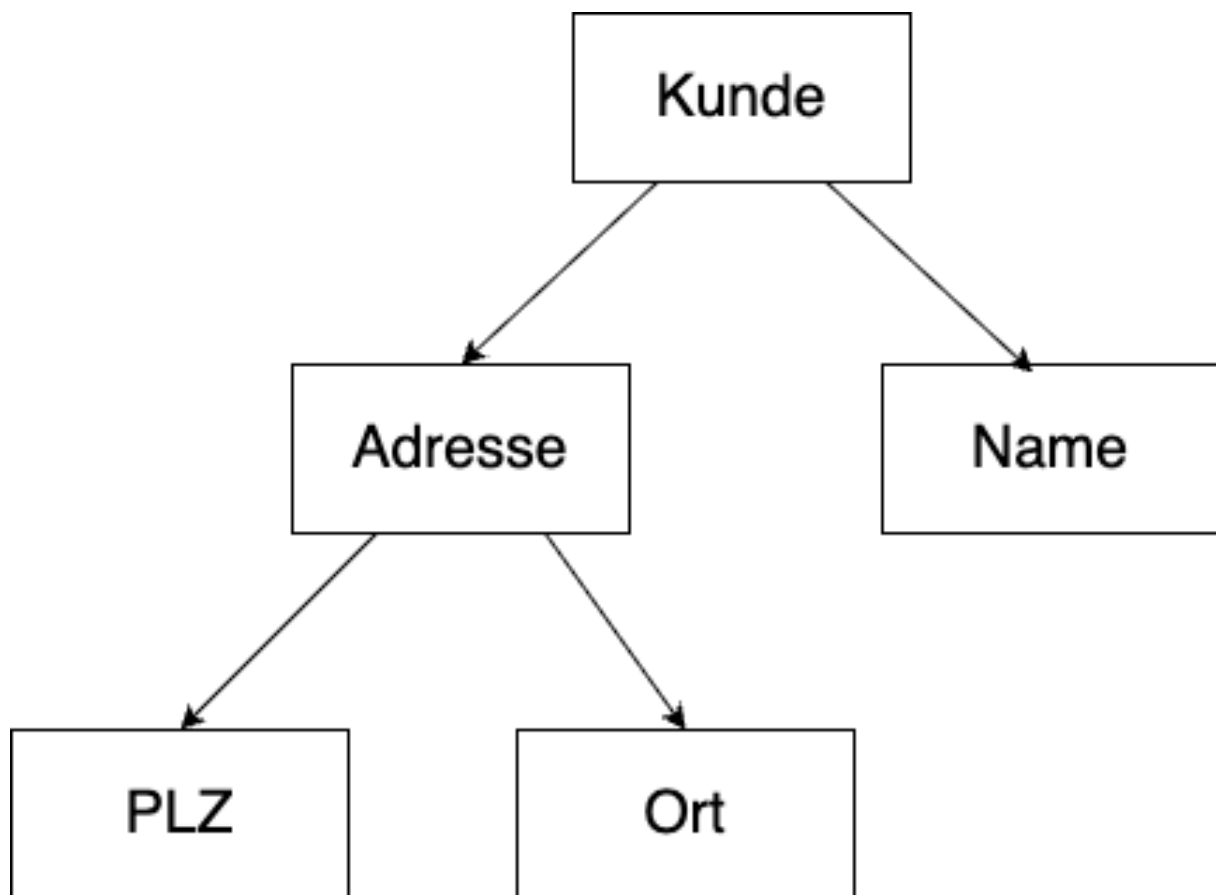


Abbildung 3. Beispiel: hierarchisches Datenbankmodell

Das hierarchische Modell erinnert an Baumstrukturen. Jede Entität hat einen Vorgänger – außer der ersten – und kann mehrere Nachfolger haben. Dadurch ist das Modell gut rekursiv durchsuchbar. Es gibt immer nur einen Einstiegspunkt.

Vorteile

- Effiziente computergerechte Datenorganisation
- Sehr schnell

Nachteile

- Benutzer muss die Datenstruktur gut kennen
- Redundanzen sind möglich

2.3 Netzwerkmodell

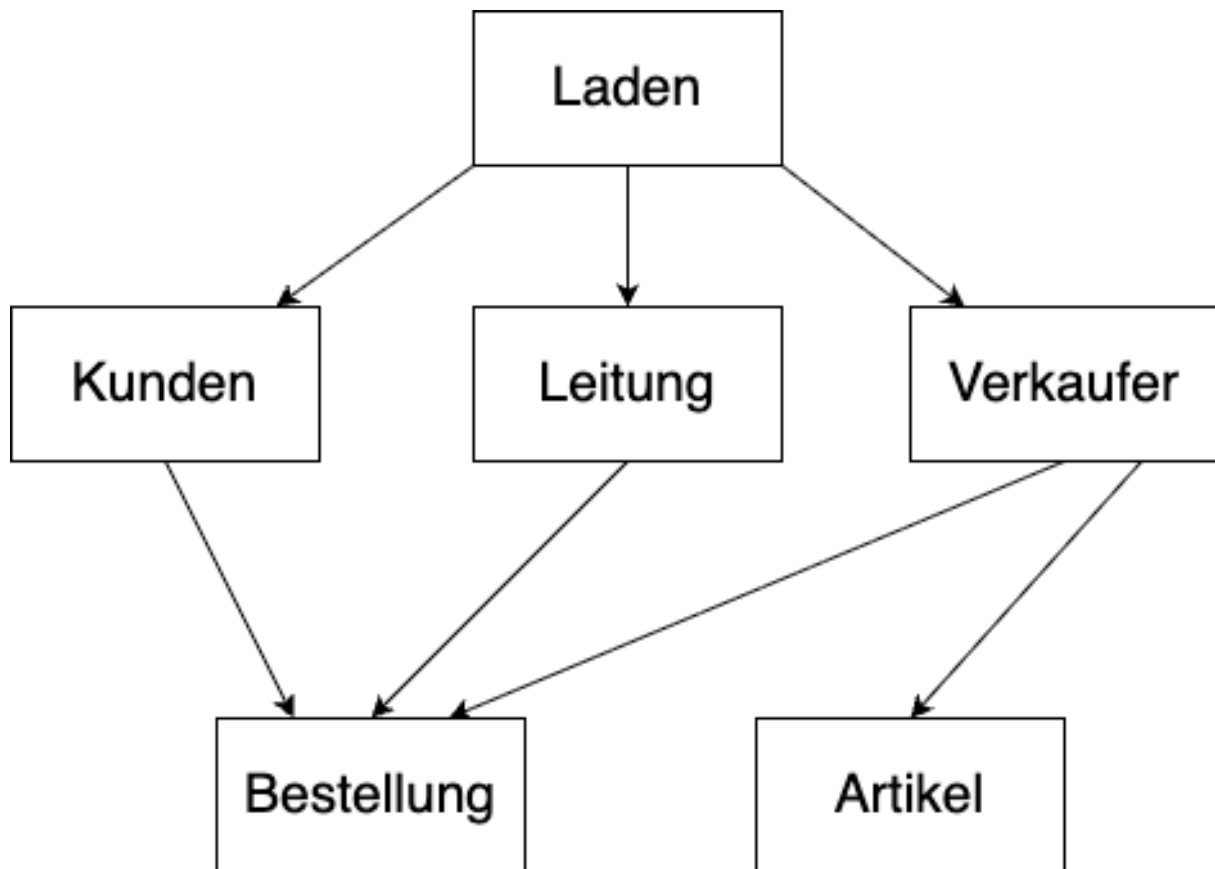


Abbildung 4. Beispiel: Netzwerk-Datenbankmodell

Das Netzwerkmodell ist nah am hierarchischen Modell, allerdings bietet es mehr als einen Einstiegspunkt. So kann man über einen Kunden an dessen Bestellung gelangen, aber auch über den Verkäufer.

Vorteile

- Flexibler Zugriff über mehrere Einstiegspunkte
- Komplexe Strukturen können abgebildet werden

Nachteile

- Benutzer muss die Datenstruktur gut kennen
- Unübersichtlich

2.4 Objektorientiertes Datenbankmodell

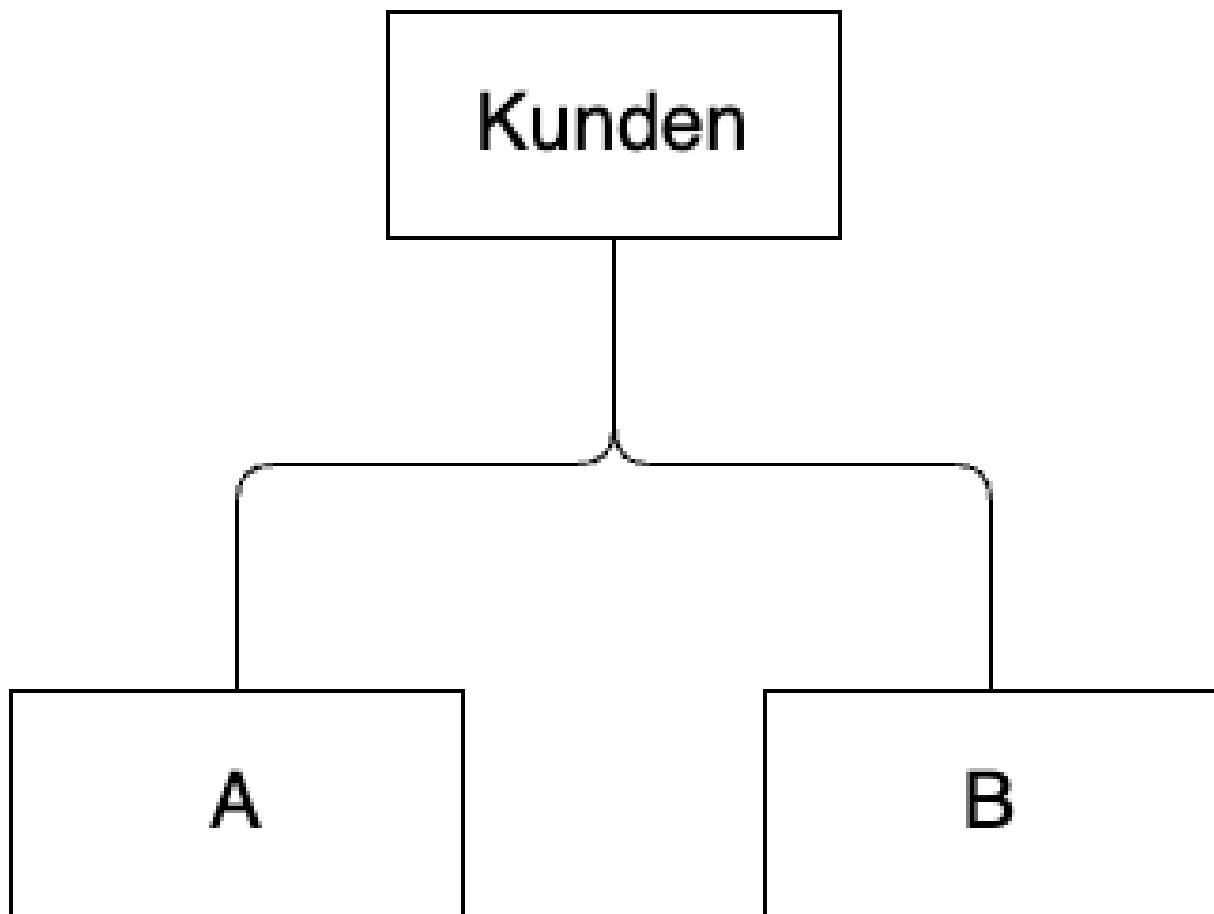


Abbildung 5. Beispiel: objektorientiertes Datenbankmodell

Bei einem objektorientierten Datenbankmodell werden Objekte ohne Zerlegung gespeichert, also als vollständige Objekte. Auch Vererbung, Kapselung und Polymorphie sind integriert. Dadurch lassen sich komplexe Datenstrukturen darstellen.

Heutzutage ist dieses Modell weitgehend obsolet, da es für relationale Datenbanken `ORM`s gibt, z. B. *Entity Framework Core* für C#.

Vorteile

- Das Problem des *Object-Relational Impedance Mismatch* wird behoben
- Komplexe Kompositionen (`JOIN`s) entfallen
- Schlüsselverwaltung systemintern

Nachteile

- Bis heute geringe Verbreitung (z. B. db4o), bei Mengenoperationen langsam
- Aufwändige Bearbeitung vererbter Objekte, inkl. Schreibsperrern

3 Datenverteilung

Logisch zusammenhängende Datenbestände werden physisch verteilt. Die Verwaltung erfolgt dabei übergeordnet durch das Datenbank-System (DB-System).

3.1 Gründe für die Datenverteilung

Die wesentlichen Gründe für die physische Verteilung von logisch zusammenhängenden Daten sind:

- **Verfügbarkeit:** Erhöhung der Systemverfügbarkeit durch Redundanz und lokale Zugriffsmöglichkeiten.
- **Sicherheit:** Schutz der Daten durch Verteilung und kontrollierte Zugriffsmöglichkeiten auf verschiedenen Knoten.
- **Effizienzsteigerungen:** Lokale Verarbeitung von Datenanfragen verringert die Netzwerklast und verbessert die Antwortzeiten.
- **Mobilität:** Unterstützung von mobilen Anwendungen und Zugriffen von verschiedenen geografischen Standorten aus.
- **Skalierbarkeit:** Einfachere Erweiterung der Systemkapazität durch Hinzufügen weiterer Knoten im verteilten System.
- **Zuverlässigkeit:** Erhöhte Ausfallsicherheit durch die Vermeidung von Single Points of Failure.

4 Datenbanken in verteilten Systemen

Verteilte Systeme

Ein verteiltes System ist eine Sammlung mehrerer unabhängiger Computer bzw. Prozesse, die nach außen hin wie ein einziges System wirken. Die einzelnen Knoten kommunizieren dabei über ein Netzwerk miteinander.

Bei verteilten Systemen und dem Zugriff auf eine Datenbank treten immer wieder Fehler auf. Häufige Problemfelder sind:

1. **Performance:** Bei verteilten Systemen kommen oft Performance-Probleme zum Tragen. Faktoren, die auf die Performance einwirken, sind die Leistung des Datenbankservers sowie der Code der Anwendung – etwa, ob ein `SQL`-Statement so geschrieben wurde, dass es die Daten effizient abfragt. Aber auch Up- und Download des Netzwerks sind wichtig. Gerade Mitarbeiter*innen, die über ein `VPN` im Netzwerk sind, erleben häufig eine starke Latenz. Bei vielen Anfragen an einen Datenbankserver verstärkt sich das noch, weil die Hardware an ihre Grenzen kommt.
2. **Verfügbarkeit:** Dadurch, dass ein Server meist viele Anfragen verarbeiten muss, ist eine Hochverfügbarkeit schwer zu erreichen. Daher wird oft darauf verzichtet, bestimmte Features in Programme einzubinden, da sonst der Server zu stark ausgelastet wird.
3. **Konsistenz:** Bei mehreren Nutzern ist es oft schwer, die Konsistenz zu halten – insbesondere, wenn mehrere Personen auf denselben Datensatz zugreifen. Beispiel: Eine Person aus dem Telefonverkauf greift auf einen Artikel zu, um dem Kunden den Preis zu nennen, während gleichzeitig eine Person aus dem Einkauf denselben Preis anpasst. Um die Konsistenz zu halten, werden oft einzelne Felder oder sogar ganze Datensätze gesperrt.

4.1 CAP-Theorem

Bevor wir uns Lösungen für diese Probleme ansehen, müssen wir das CAP-Theorem verstehen. Dieses wurde im Jahr 2000 von *Eric Brewer* aufgestellt und 2002 von *Seth Gilbert* und *Nancy Lynch* formal bewiesen. Das Theorem besagt, dass man Konsistenz (*Consistency*), Verfügbarkeit (*Availability*) und Partitionstoleranz (*Partition Tolerance*) nie gleichzeitig vollständig erreichen kann. Daher muss man immer abwägen, was wichtiger ist.

CAP-Theorem

Von den drei Eigenschaften Konsistenz, Verfügbarkeit und Partitionstoleranz sind in einem verteilten System stets nur zwei zugleich vollständig erreichbar.

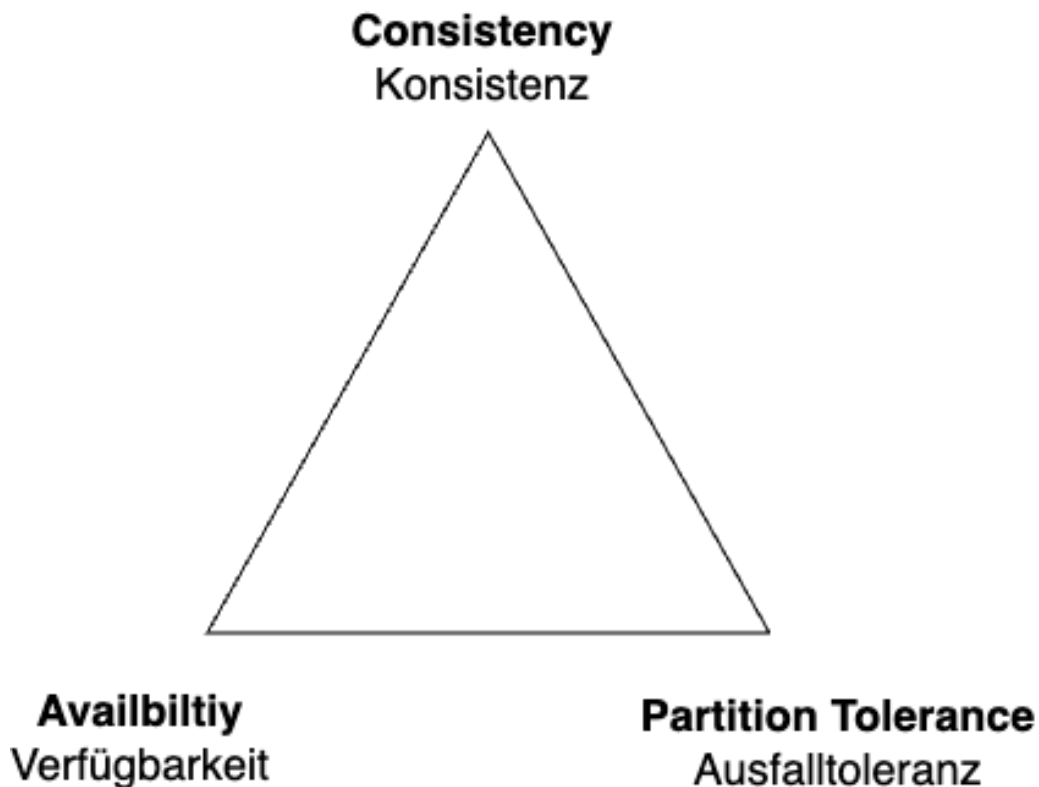


Abbildung 6. CAP-Theorem

4.2 Zentrale Datenbank

Bei einer zentralen Datenbank hat man einen zentralen `SQL`-Server, auf dem immer alle Abfragen laufen. Dadurch ist eine gewisse Konsistenz gewährleistet. Das Problem dabei ist jedoch, dass der `SQL`-Server ausgelastet werden kann, was sich auf die Performance auswirkt – dadurch wird alles langsamer, und auch die Ausfallsicherheit ist schwierig. Allerdings kann ein Server auch stark ausgestattet sein, sodass dies weniger ein Problem darstellt. Bei einem zentralen System muss man die Konfiguration vorab festlegen und kann später schlecht skalieren.

4.3 Replikation

Statt nur einen Server zu haben, nutzt man mehrere und repliziert die Daten. Damit hat man einen konsistenten Datenbestand, kann aber Anfragen und Last verteilen.

4.3.1 Arten

Primary / Secondary

Es gibt einen Hauptserver (*Primary*); die Replikationen werden auf andere Server kopiert (*Secondary*). Schreibzugriffe gibt es nur auf dem Primary, Leseanfragen können hingegen auch an die Secondary-Server geleitet werden.

Peer to Peer (Multi-Master)

Alle Nodes sind gleichberechtigt und erlauben Schreibzugriffe. Änderungen müssen daher immer auf allen Nodes synchronisiert werden.

4.3.2 Modus**Asynchrone Replikation**

Die Änderung ist sofort abgeschlossen und wird erst später repliziert. Die Performance der Schreiboperation ist gut, dafür ist der Datenbestand vorübergehend inkonsistent.

Synchrone Replikation

Die Änderung ist erst nach der Synchronisation abgeschlossen. Die Performance der Schreiboperation ist langsamer, dafür ist der Datenbestand durchgehend konsistent.

4.4 Sharding

Beim *Sharding* werden zusammengehörige Daten-Aggregate jeweils einem Node zugeordnet. Schreibzugriffe für einen Datensatz erfolgen immer nur über diesen einen, definierten Node. Dadurch sind Lese- und Schreiboperationen skalierbar, und durch die Daten-Lokalität ergibt sich eine bessere Latenz. Außerdem ist das System konsistent: Da Schreibzugriffe für einen Datensatz nur über einen Node laufen, entstehen keine Konflikte.

Sharding vs. Replikation

Bei der *Replikation* liegen dieselben Daten auf mehreren Nodes. Beim *Sharding* wird der Datenbestand aufgeteilt – jeder Node hält nur einen Teil der Daten.

Mittels Sharding können umfangreiche Datenmengen verwaltet werden, welche die Kapazitäten eines einzelnen Servers sprengen würden.

Nachteil

Der größte Nachteil des Shardings ist, dass ein Zugriff über andere Kriterien als das Aufteilungskriterium unverhältnismäßig aufwändig ist. Abfragen über andere Kriterien oder `JOINS` müssen im Allgemeinen auf alle Server aufgeteilt werden. Datenmodell und Zugriffspfade müssen daher so entworfen werden, dass derartige Zugriffe nur selten oder gar nicht vorkommen – sonst werden die Vorteile des Shardings zunichtegemacht.

Aufgrund dieser Eigenschaften ist Sharding ideal für `NoSQL`-Datenbanken.

4.5 Caching

Beim *Caching* verwalten die Clients eine eigene Kopie der Daten (oder von Teilen davon).

Vorteile

- Schneller, lokaler Zugriff
- Entlastung des Netzwerks und der Server

Nachteile

- Konsistenz – wie lange sind die gecachten Daten gültig?
- Bei Schreiboperationen im Cache sind Konflikte möglich

Beispiel: Caching in der Praxis

DNS, Browser-Cache, Offline-Webanwendungen sowie Microsoft BranchCache und Windows Updates.

4.6 Map/Reduce

Map/Reduce ist ein Programmiermodell zur Parallelisierung, das häufig in Software-Frameworks realisiert ist – etwa in Hadoop oder MongoDB. Komplexe Aufgaben werden dabei in zwei Phasen bearbeitet: *Map* und *Reduce*.

Divide and Conquer

Map/Reduce basiert auf dem Prinzip *Divide and Conquer* („Teile und herrsche“): Ein großes Problem wird rekursiv in kleinere, unabhängige Teilprobleme zerlegt, die einzeln gelöst werden. Anschließend werden die Teilergebnisse wieder zu einer Gesamtlösung zusammengefügt. Weil die Teilprobleme unabhängig voneinander sind, können sie parallel bearbeitet werden.

1 Map: Die Gesamtaufgabe wird in einzelne Arbeitspakete aufgeteilt und auf verschiedene Nodes verteilt. Jeder Node bearbeitet seine Teilaufgabe unabhängig von den anderen – dadurch ist eine parallele Bearbeitung möglich.

2 Reduce: Die Teilergebnisse der einzelnen Nodes werden wieder zu einem Gesamtergebnis zusammengefügt.

Kernidee

Durch das rekursive Aufteilen in unabhängige Teilaufgaben (*Divide and Conquer*) lässt sich die Bearbeitung über viele Nodes parallelisieren und damit auf großen Datenmengen skalieren.

5 Managementunterstützungssysteme

Hinter dem Begriff Managementunterstützungssystem steckt – für alle unter 50 – im Grunde *lobotomized Data Science*, also das Extrahieren von Wissen aus großen Datenmengen, nur ohne Betrachtung der Zukunft und mit einem stärkeren Fokus auf Berichte über vergangene Daten.

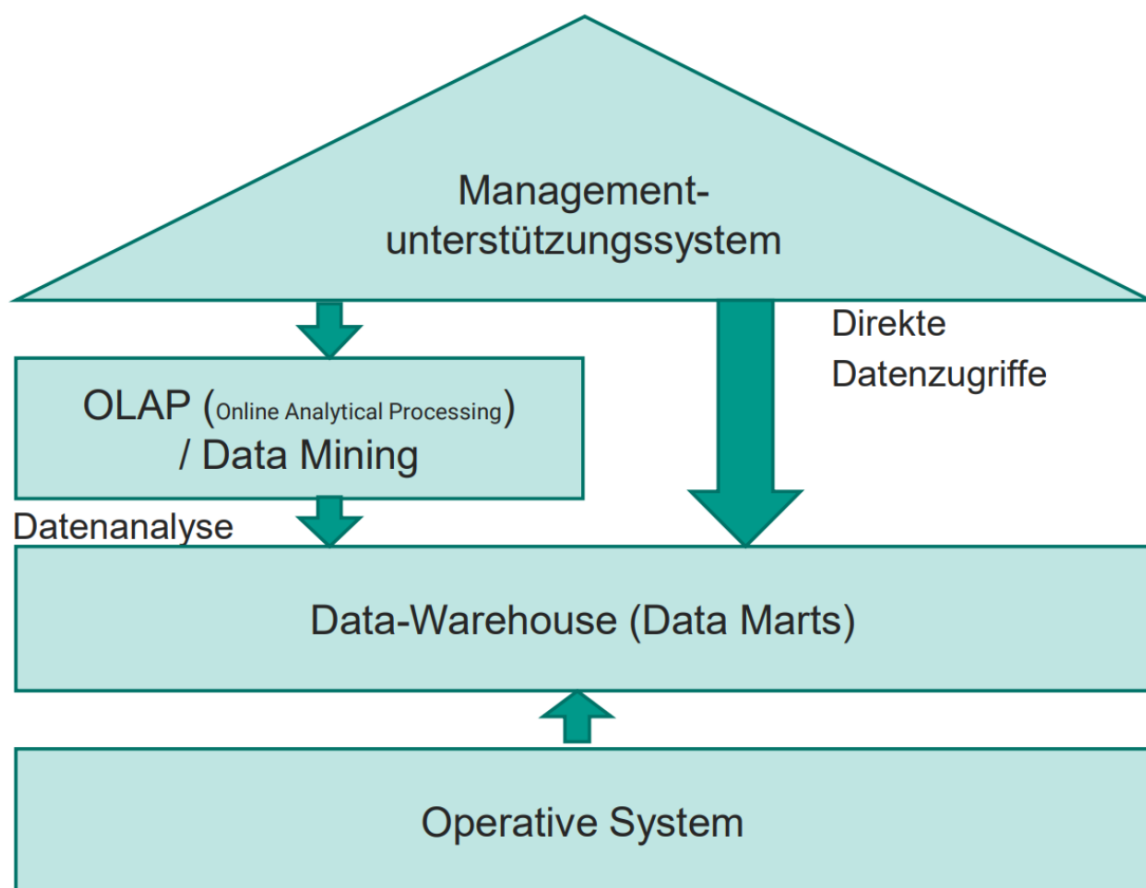


Abbildung 7. Aufbau eines Managementunterstützungssystems

5.1 Data Warehouse

Data Warehousing beschreibt ein Konzept zur Informationsanalyse, -selektion und -aufbereitung, bei dem dem Anwender entscheidungsrelevante Daten aus unterschiedlichen Quellen in einer einheitlichen, zentralen Systemumgebung zur Auswertung zur Verfügung gestellt werden.

- Ein Data Warehouse ist eine extrem große Datenbank, die unabhängig von den operativen Systemen installiert ist und ein vordefiniertes Set von meist verdichteten Daten enthält.
- Die Benutzer können auf die Datenbank beliebig zugreifen und alle gewünschten Daten herausziehen, sofern die Informationen enthalten sind und eine Zugriffsberechtigung vorliegt.
- Der Grad der Informationstiefe und die jeweilige Sichtweise können dabei indivi-

duell festgelegt werden.

- Betrachtungsebenen sind beispielsweise Konten, Kostenstellen, Kostenträger, Artikel, Kunden, Märkte oder Vertreter. Es gibt keine festen Vorgaben für die Art der Reihenfolge und der Verdichtung.

5.2 Data Mart

Data Marts sind abteilungs- oder funktionsbezogene Datenbanken mit einer speziellen Aufgabenstellung, die eine Untermenge der im zentralen Data Warehouse gespeicherten operativen Daten enthalten.

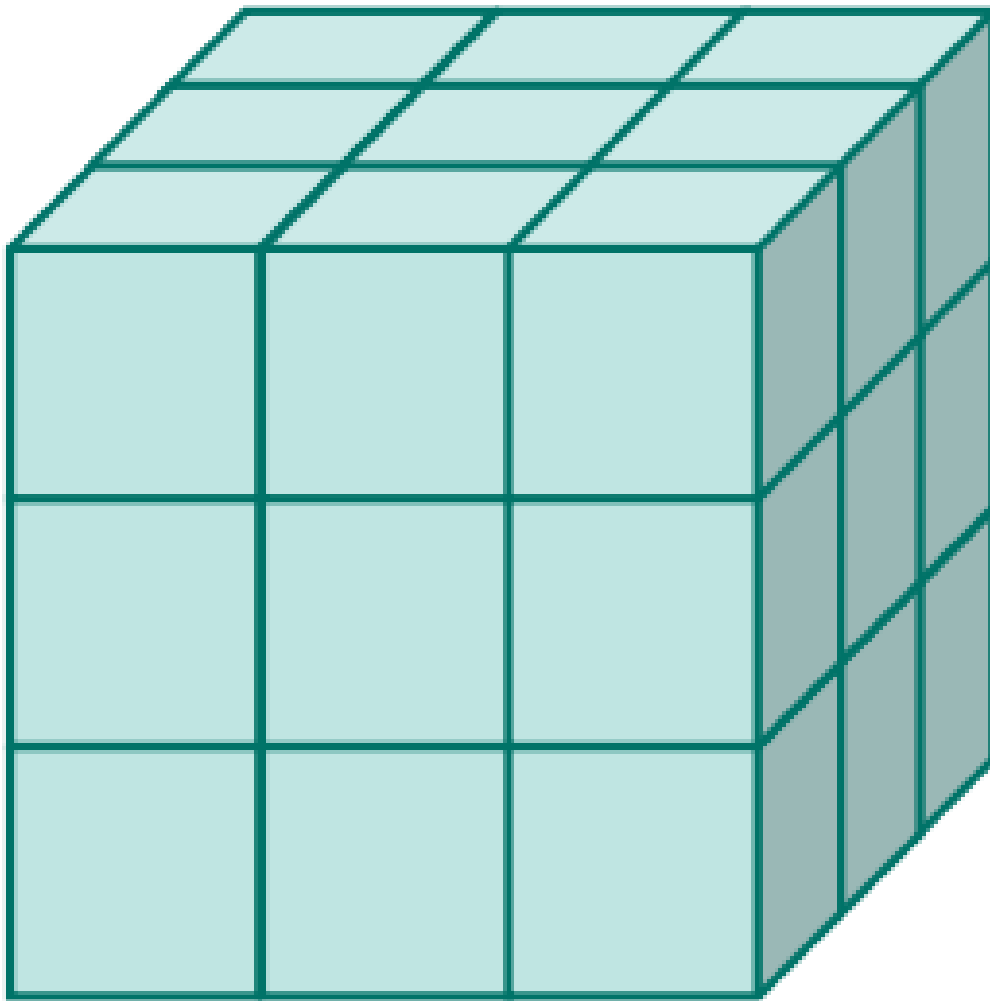


Abbildung 8. Data Mart

- Data Marts enthalten lediglich Informationen, die in einem bestimmten Funktionsbereich, z. B. Marketing, des Unternehmens zur Anwendung kommen.
- Data Marts bilden daher eine Teilmenge der umfassenden Data-Warehouse-Datenbank.
- Data Marts werden auch Datenwürfel, Power Cube oder Info Cube genannt.

5.3 OLTP und OLAP

Das „Online“ bedeutet, dass auf Eingaben von Anwender*innen quasi (so gut wie möglich) in Echtzeit reagiert wird. Beide Begriffe hören sich ähnlich an, beschreiben aber zwei getrennte Welten: **OLTP** beschäftigt sich damit, wie Business-Funktionen und Anwendungen ihre Daten in einer Anwendungsdatenbank speichern. Die Daten, die durch OLTP entstanden sind, befüllen das Data Warehouse, in dem mit **OLAP** Auswertungen durchgeführt werden. Wichtig ist hier die Trennung zwischen den beiden Bereichen – und dass man für die unterschiedlichen Prozesse eine unterschiedliche Datenmodellierung nutzt.

Zwei getrennte Datenbanken

Die Data-Warehouse-Datenbank ist **nicht** die Anwendungsdatenbank, sondern eine eigene, zweite Datenbank mit eigenem Datenmodell: Die Anwendungsdatenbank (OLTP) ist normalisiert und schreiboptimiert, das Data Warehouse (OLAP) ist dimensional und denormalisiert modelliert (Star-Schema) und damit leseoptimiert.

5.3.1 Online Transaction Processing – OLTP

Online Transaction Processing beschäftigt sich mit der Verarbeitung von Transaktionen. Eine Transaktion wäre hier z. B. eine Online-Shop-Bestellung. Ziel: möglichst schnellen Zugriff auf Business-Objekte wie Artikel zu haben, um diese schnell verändern und aktualisieren zu können. OLTP beschäftigt sich also vor allem mit dem **Schreiben** von Daten – die Anwendungsdatenbank ist dafür normalisiert (3NF), damit jede Änderung nur an einer Stelle passieren muss.

5.3.2 Online Analytical Processing – OLAP

Online Analytical Processing ist ein „top-down“-Ansatz, um sehr flexibel mehrdimensionale Datenanalysen durchzuführen und Hypothesen zu bilden. Ziel: Daten verarbeiten und auswerten. OLAP beschäftigt sich mit dem **Lesen** von Daten und läuft gegen die Data-Warehouse-Datenbank – nicht gegen die Anwendungsdatenbank.

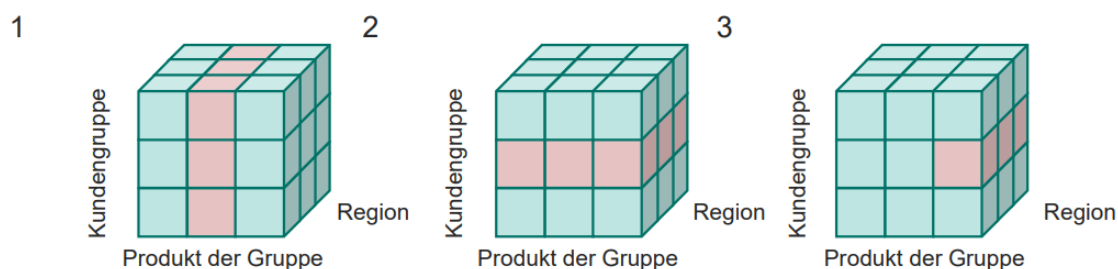


Abbildung 9. Datenanalyse mit OLAP

5.3.3 Dimensionen und Fakten

Die Grundidee der mehrdimensionalen Analyse: Man trennt die Daten in **Fakten** und **Dimensionen**.

- **Fakten (Kennzahlen):** die Zahlen, die ausgewertet werden sollen – z. B. Umsatz oder verkaufte Menge.

- **Dimensionen:** die Blickwinkel, aus denen man eine Kennzahl betrachtet – z. B. Zeit, Produkt, Kunde oder Region.
- Stellt man sich die Dimensionen als Achsen vor, entsteht der *Datenwürfel* (Cube): Jede Zelle des Würfels enthält eine Kennzahl am Schnittpunkt der Dimensionen.
- Dimensionen haben Hierarchien (z. B. Tag → Monat → Quartal → Jahr), über die sich Kennzahlen verdichten oder verfeinern lassen.

Beispiel: Drei Dimensionen, eine Kennzahl

„Wie hoch war der **Umsatz** von Produkt `Pizza Salami` in der Region `Süd` im `März 2026`?“ – Die Kennzahl ist der Umsatz, betrachtet über die drei Dimensionen Produkt, Region und Zeit.

5.3.4 Star-Schema

Das Star-Schema ist das typische Datenmodell für die Data-Warehouse-Datenbank – und genau hier zeigt sich, dass sie anders modelliert wird als die Anwendungsdatenbank:

- In der Mitte steht die **Faktentabelle**: Sie enthält die Kennzahlen sowie Fremdschlüssel auf alle Dimensionstabellen.
- Sternförmig darum liegen die **Dimensionstabellen**: Sie enthalten die beschreibenden Attribute einer Dimension (z. B. bei der Zeit: Tag, Monat, Quartal, Jahr).

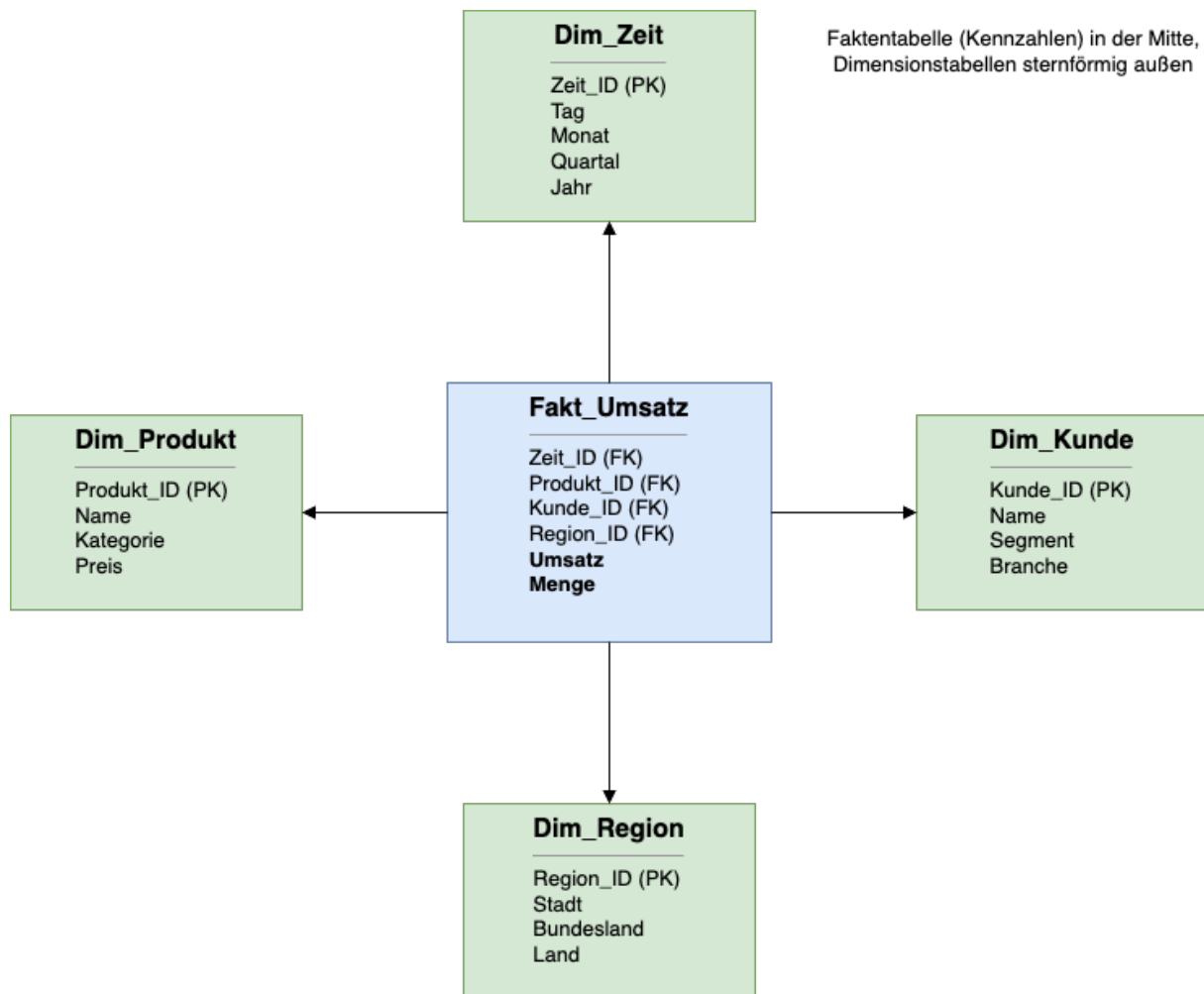


Abbildung 10. Star-Schema: Faktentabelle mit sternförmig angeordneten Dimensionstabellen

Bewusster Verstoß gegen die Normalisierung

Die Dimensionstabellen sind absichtlich **denormalisiert** (z. B. steht das Jahr redundant in jeder Zeile von `Dim_Zeit`). Die Redundanz wird in Kauf genommen, damit Auswertungen mit möglichst wenigen Joins auskommen und schnell lesbar sind. Da das Data Warehouse nur befüllt und gelesen, aber kaum geändert wird, sind die üblichen Update-Anomalien hier kein Problem.

Merkmal	OLTP	OLAP
Zweck	Tagesgeschäft abwickeln	Analysen und Berichte
Zugriff	Schreiben und Ändern	Lesen
Datenbank	Anwendungsdatenbank	Data-Warehouse-Datenbank
Datenmodell	normalisiert (3NF)	Star-Schema (denormalisiert)
Datenmenge pro Zugriff	einzelne Datensätze	große, verdichtete Datenmengen
Nutzer	Anwendungen, Endnutzer	Management, Analyst*innen

5.4 Exkurs: CQRS

Nicht klausurrelevant

Dieser Abschnitt ist **nicht klausurrelevant**. Er zeigt aber, dass die Idee der OLTP/OLAP-Trennung auch auf Anwendungsebene wieder auftaucht.

Command Query Responsibility Segregation (CQRS) ist ein Architekturmuster, das Schreiben und Lesen innerhalb einer Anwendung strikt trennt:

- **Commands** (Schreiben) gehen an ein eigenes **Write-Modell**, das den Zustand validiert und ändert – dahinter liegt typischerweise eine normalisierte Schreib-Datenbank.
- **Queries** (Lesen) gehen an ein eigenes **Read-Modell**, das nur liest – dahinter liegt eine denormalisierte Lese-Datenbank mit fertig aufbereiteten Views.
- Die beiden Datenbanken werden über *Events* synchronisiert. Das Read-Modell hinkt dem Write-Modell dabei minimal hinterher (*eventual consistency*).

Wie bei OLTP und OLAP gilt: Zwei unterschiedliche Aufgaben (schnell schreiben vs. schnell lesen) bekommen zwei unterschiedlich modellierte Datenbanken.

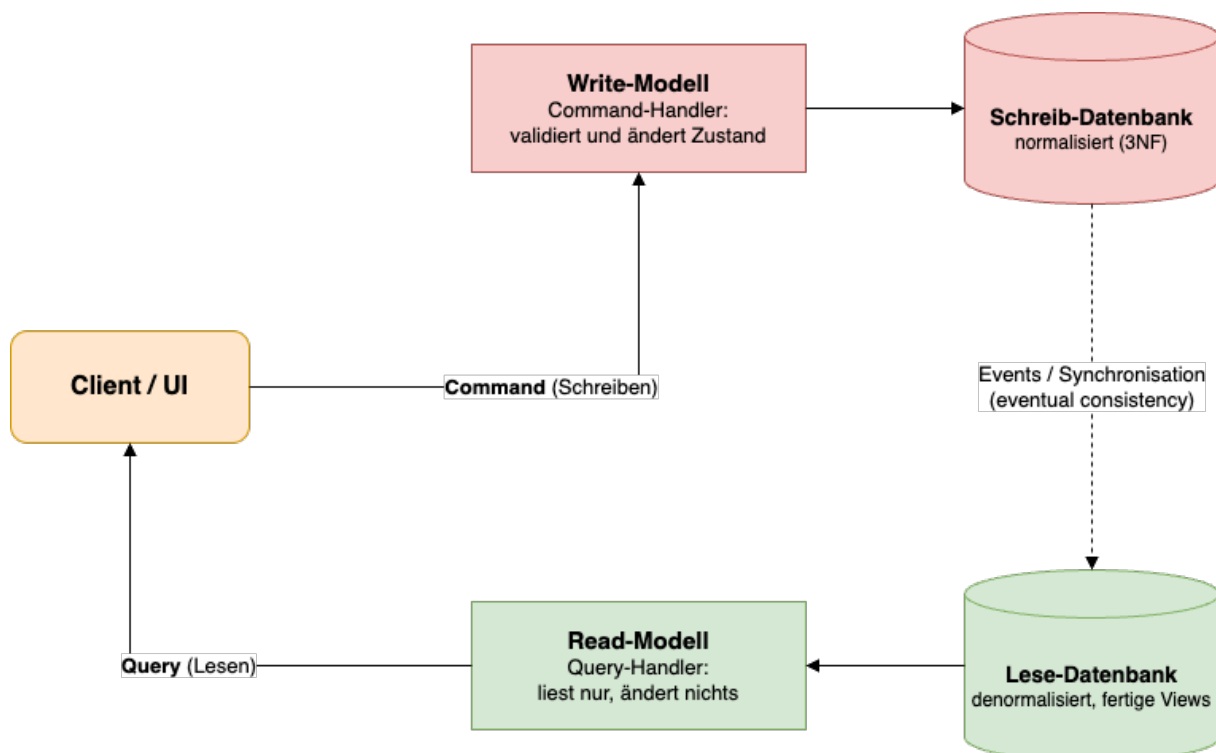


Abbildung 11. CQRS: getrennte Wege für Commands (Schreiben) und Queries (Lesen)

5.5 Data Mining

Data Mining bezeichnet das automatische Entdecken von Modellen und Mustern in großen Datenbeständen durch Anwendung bestimmter Data-Mining-Werkzeuge.

- Data Mining ist eine „intelligente“ Anwendung auf Basis einer Data-Warehouse-Architektur.

- Beziehungen zwischen Daten werden hergestellt (Zeitreihenanalyse, Datenklassifikation, Prognosen).
- Data Mining versucht, den Anwender auf bemerkenswerte Datenkonstellationen aufmerksam zu machen und zu signifikanten Aussagen hinzuführen.
- Analyseziel: „Finde Gold in deinen Daten!“

6 Datenbank-Architektur

6.1 ANSI-3-Ebenen-Modell

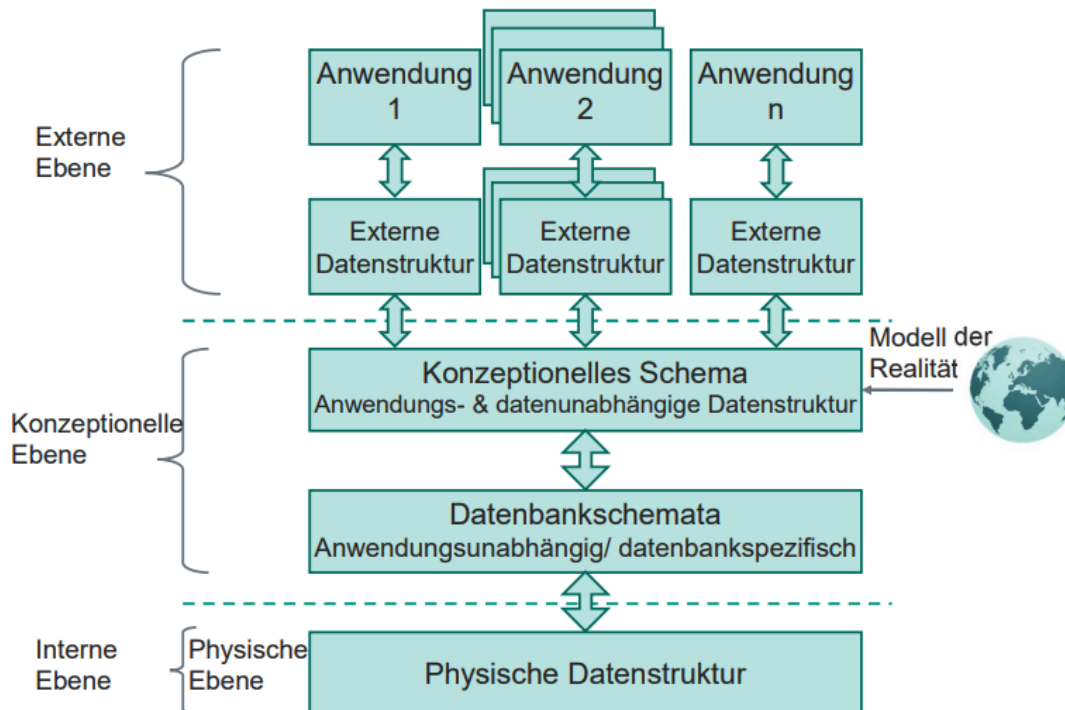


Abbildung 12. Aufbau des ANSI-3-Ebenen-Modells

Die ANSI-3-Ebenen-Architektur gibt es seit den 1970er-Jahren. *ANSI* steht für *American National Standards Institute* und ist damit in etwa mit dem deutschen DIN vergleichbar. Die 3-Ebenen-Architektur wird auch *SPARC* genannt, abgeleitet von *Standards Planning And Requirements Committee* – einem Ausschuss innerhalb von ANSI, von dem die Architektur stammt.

ANSI/SPARC-Architektur

Die 3-Sichten-Architektur ist die Grundlage einer konsistenten, modularen, interoperablen, wartbaren und skalierbaren Datenbank. Sie trennt die Daten in eine **interne**, eine **konzeptionelle** und eine **externe** Ebene.

6.1.1 Interne Ebene

Die interne Ebene ist das Fundament einer Datenbank. Hier liegt ein Datenbankmodell zugrunde, wie es im Kapitel *Datenbankmodelle* besprochen wird. Auf dieser Ebene werden die Aspekte der physischen Speicherung festgelegt – also *wie* und *wo* die Daten konkret abgelegt werden.

6.1.2 Konzeptionelle Ebene

Die konzeptionelle Ebene ist das Bindeglied zwischen der internen und der externen Ebene. Hier werden die Daten modelliert, beispielsweise über ein ER-Diagramm. Auf dieser

Ebene wird das Datenbankschema festgelegt, ebenso das konzeptionelle Schema für `Primary Key`, `Foreign Key` sowie sonstige Regeln zur Sicherung der Datenkonsistenz.

6.1.3 Externe Ebene

Die externe Ebene definiert, wie Nutzer auf die Datenbank zugreifen. Dazu zählen User Interfaces, Anwendungs-APIs, Views sowie – im Fall einer API – das Äquivalent in Form von *Data Transfer Objects* (`DTO`s). Hier können außerdem Plausibilitätsprüfungen und sonstige Integrationsregeln festgelegt werden. Auf dieser Ebene findet der `SQL`-Zugriff statt.

6.2 Datenbank-Entwurfsphasen

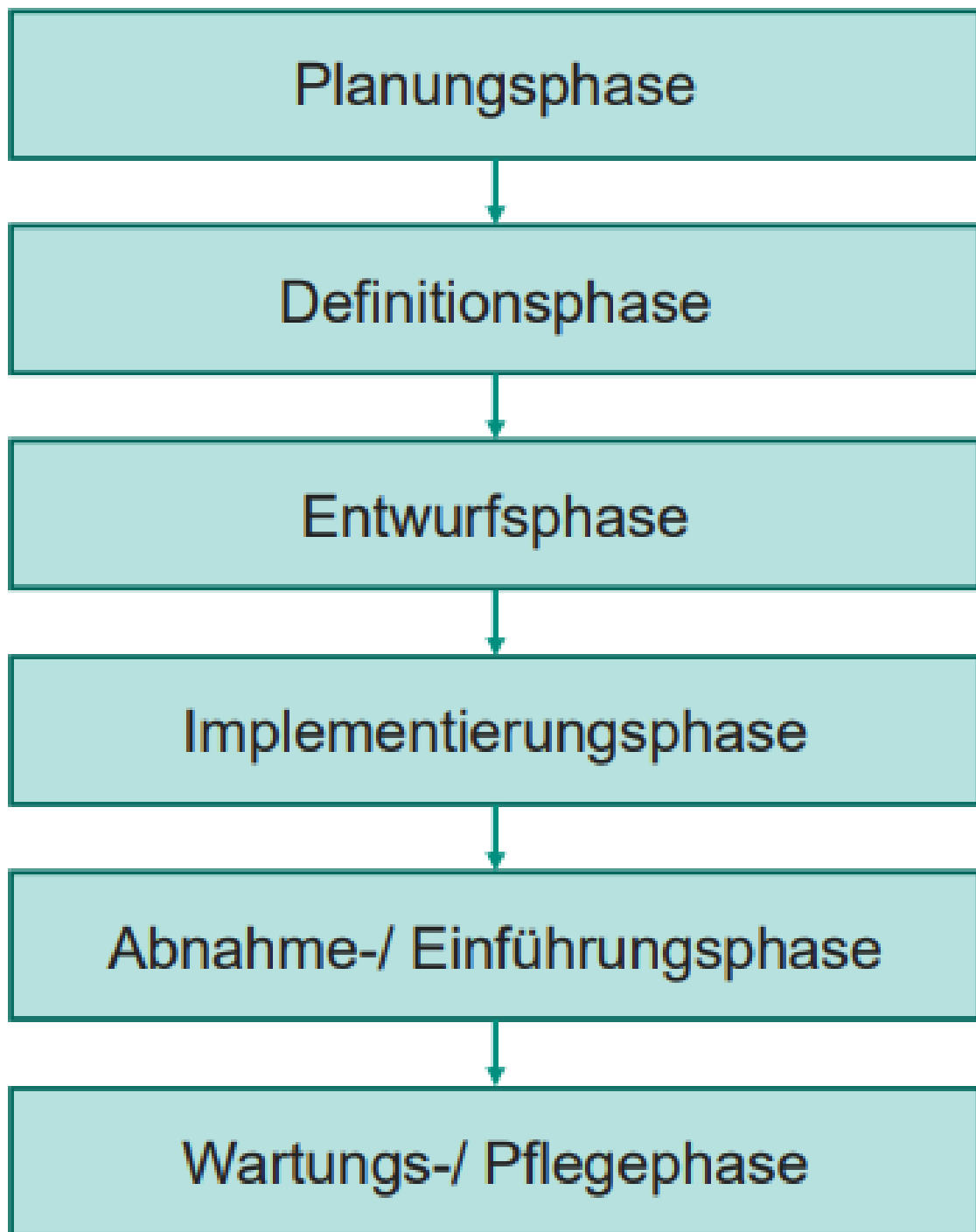


Abbildung 13. Entwurfsphasen einer Datenbank

Der Entwurf einer Datenbank lässt sich in sechs Phasen unterteilen.

6.2.1 Planungsphase

In der Planungsphase geht es um die grundlegenden Festlegungen – also darum, was überhaupt in der Datenbank gespeichert werden soll. Beispielsweise könnte ein Projekt darin bestehen, die Touren eines Logistikunternehmens zu speichern.

6.2.2 Definitionsphase

Hier wird festgelegt, welche Anforderungen die Datenbank erfüllen muss und welche Daten wo und wie erfasst werden müssen.

6.2.3 Entwurfsphase

Hier werden die technischen Entscheidungen getroffen – etwa, welches Datenbankmodell genutzt wird. Außerdem entsteht in dieser Phase das ER-Diagramm.

6.2.4 Implementierungsphase

Hier wird der Entwurf in Code umgesetzt, meist als `DDL` in `SQL`. Eventuell werden auch Views angelegt und vielleicht sogar eine API.

6.2.5 Abnahme- und Einführungsphase

Hier werden die Datenbank bzw. die umgebende Software eingeführt. Anschließend wird geprüft, ob die Anwender mit der Anwendung zufrieden sind.

6.2.6 Wartungs- und Pflegephase

Hier werden mögliche Fehler behoben; auch eventuelle Erweiterungen fallen in diese Phase.

7 NoSQL

NoSQL steht für *Not Only SQL* und bietet eine Alternative zu relationalen Datenbank-Management-Systemen.

Das liegt insbesondere daran, dass relationale Datenbanksysteme bei flexiblen Datenstrukturen (etwa für Konfigurationen), hoher Schreiblast, globalen Netzwerken und sehr großen Datenmengen an ihre Grenzen kommen - gerade bei Echtzeitanalysen oder **SaaS**-Anwendungen. Auch wenn man an die 5 V von Big Data denkt, ist NoSQL für Varietät wichtig.

Eine andere Sache, die man bei NoSQL im Kopf behalten sollte, ist, dass eine NoSQL-Datenbank oder eine relationale Datenbank nie die wirkliche Lösung ist, sondern man für unterschiedliche Daten unterschiedliche Datenbanksysteme nutzt.

Die Ziele und die Motivation von NoSQL sind:

1. ein einfacheres Design
2. einfache Skalierung
3. Kontrolle über die Daten
4. verteilte Architektur – eine passende Datenbank für jeden Use Case
5. größere Datenbanken für Big Data
6. horizontale Skalierung in der Cloud
7. flexible Datenstruktur
8. hohe Schreiblast

7.1 Modelle

7.1.1 Key-Value

In einem Key-Value-Store – wie beispielsweise einem **Redis**-Cache – werden Daten als Schlüssel-Wert-Paare (*Key-Value-Pairs*) gespeichert. Ein Key-Value-Modell eignet sich gut für temporäre Daten.

7.1.2 Document Store

Daten werden in Dokumenten gespeichert. Ein Beispiel ist **MongoDB**, das mit **JSON**-Dateien arbeitet. MongoDB ist besonders gut für Konfigurationsdateien geeignet.

7.1.3 Column Store

Bei einem Column-Store existieren keine festen Schemas, da jede Zeile andere Spalten besitzen kann.

7.1.4 Graphdatenbank

Eine Graphdatenbank stellt Daten als Graphen dar. Die Verknüpfung über Knoten ermöglicht die Darstellung von Netzwerken und Wortzusammenhängen und bildet so die Grundlage für **LLMs** und Big Data. Ziel ist es, Relationen abzubilden.

8 Datenmodellierung

Datenmodellierung bedeutet, alle relevanten Informationen eines Systems zu beschreiben – mit einer Syntax, die einfach, aber formal und exakt genug ist. Sie ist die Schnittstelle zwischen der Realität und dem konzeptionellen Modell.

8.1 Stufen der Datenmodellierung

- **Konzeptionell:** Fachliche Beschreibung, Identifikation von Entitäten, Attributen und Beziehungen – meist als ER-Diagramm – aus den bestehenden Daten. Die Phase ist rein konzeptionell, deshalb werden noch keine Datentypen genutzt. Es wird darauf geachtet, wo Daten anfallen und welche Daten relevant sind.
- **Logisch:** Anhand des ER-Diagramms Tabellen erstellen, Festlegung von Primär- und Fremdschlüsseln, Normalisierung. Erstellung eines relationalen Schemas.
- **Physikalisch:** Umsetzung der Tabellen in einem konkreten DBMS: Datentypen, Indizes, Speicherstrukturen. Daraus folgt ein implementiertes Schema – also auch SQL-„Code“.

8.2 Entity-Relationship-Modell

Ein Entity-Relationship-Modell, kurz `ERM`, ist ein konzeptioneller Entwurf einer Datenbank. Das ERM kommt bei der semantischen Modellierung einer Datenbank zum Einsatz. Es hilft dabei, Entitäten, deren Attribute und Beziehungen zu bestimmen. Die Notation, die für ERM der Standard ist, wird *Chen-Notation* genannt. Diese wurde von Peter Chen erfunden.

8.2.1 Chen-Notation

Die Chen-Notation ist besonders beliebt, weil sie Regeln hat, wie sie erstellt und gelesen wird.

- Bei einem ERM gibt es eine feste Leserichtung: von links nach rechts und von unten nach oben.
- Bipartiter ungerichteter Graph: Kanten gibt es nur zwischen Entitäten und Beziehungen oder Entitäten und Attributen. Die Kanten gelten zusätzlich in beide Richtungen.
- Jede Entität hat ein identifizierendes Attribut → Vorbereitung für den `Primary Key`.

Die Chen-Notation besteht aus verschiedenen Elementen:

Symbol	Element	Beschreibung
	Objekt / Objekttyp	Objekt: ein konkretes Exemplar (z. B. Lehrer „Fröhlich“). Objekttyp: die Klasse gleichartiger Objekte (z. B. Lehrer).
	Eigenschaft / Eigenschaftswert	Eigenschaft: Merkmal aller Objekte. Eigenschaftswert: konkrete Ausprägung.
	Beziehung / Beziehungstyp	Beziehung: Zusammenhang zwischen zwei Objekttypen. Beziehungstyp: alle gleichartigen Beziehungen.

Jede Beziehung besitzt außerdem eine **Kardinalität**, die angibt, wie viele Objekte eines Typs an der Beziehung teilnehmen können.

Kardinalität	Beispiel
1:1	Ein Beamer befindet sich in genau einem Raum (und umgekehrt).
1:n	Ein Auftrag besteht aus n Auftragspositionen; eine Position gehört zu genau einem Auftrag.
n:m	Ein Student nimmt an mehreren Klausuren teil; an einer Klausur nehmen mehrere Studenten teil.

In den Vorlesungen haben wir als Beispiel einen Pizza-Service behandelt.

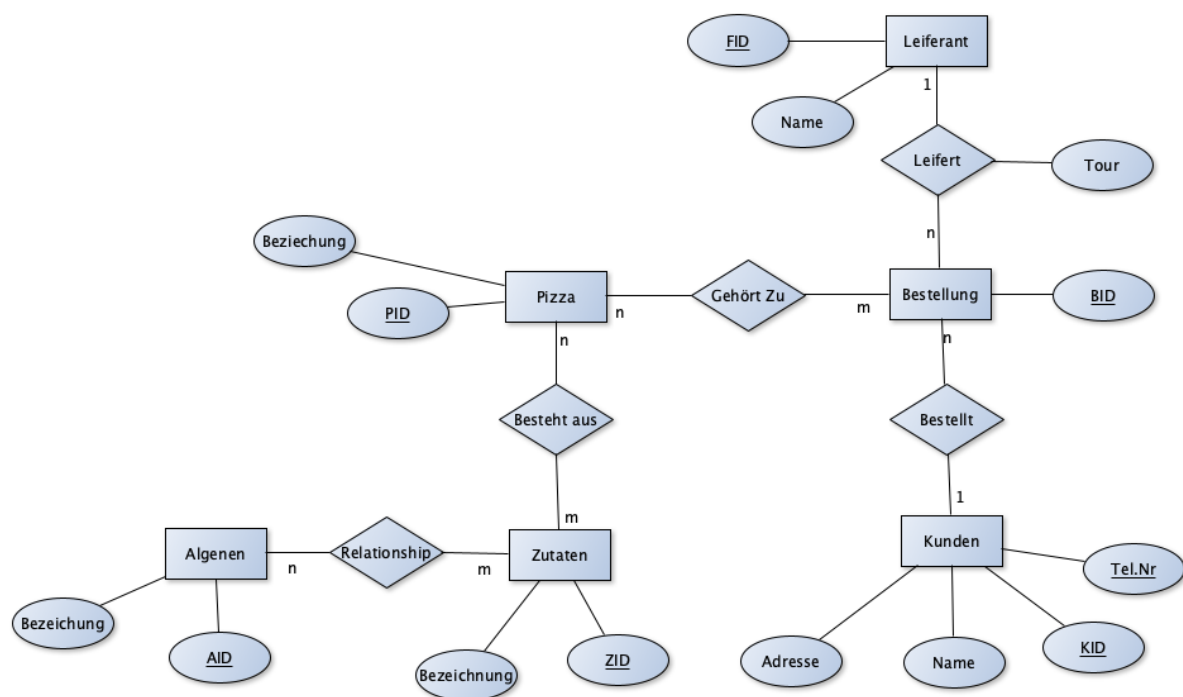


Abbildung 14. Beispiel ERM

8.3 Vom ERM zum RDM

Das *relationale Datenbankmodell* (RDM) ist Teil der logischen Modellierung: Die Entitäten des ERM werden in Tabellen bzw. Relationen überführt – samt ihren Beziehungen

und Attributen. Besonders wichtig sind dabei Primär- und Fremdschlüssel, denn sie gewährleisten die Einzigartigkeit der Zeilen und die Verknüpfung der Tabellen. Ein fertiges RDM liegt in der dritten Normalform vor.

- 1** Tabellen schaffen: die Entitäten werden zur Bezeichnung, die Attribute zu Spalten.
- 2** Das identifizierende Schlüsselattribut wird zum `Primary Key`.
- 3** Die Beziehungen betrachten und über `Foreign Keys` bzw. Beziehungstabellen im RDM umsetzen.

Kardinalität	Umsetzung im RDM
1:1	Auf einer der beiden Seiten wird der FK der anderen Entität eingetragen – am besten auf der Seite, die stärker abhängig ist.
1:n	Auf der n-Seite fügt man den FK ein (also den PK der Entität auf der 1-Seite).
n:m	Man schafft eine zusätzliche Beziehungstabelle, die beide PKs als FKs enthält.

8.4 UML-Klassendiagramm

Neben dem ERM lässt sich ein System auch objektorientiert modellieren – mit einem *UML-Klassendiagramm*. Auf Klassen und Objekte gibt es dabei zwei Sichten:

- **Sicht A:** Eine Klasse ist ein Bauplan, aus dem Objekte gebaut werden.
- **Sicht B:** Objekte werden aufgrund von Ähnlichkeiten zu einer Klasse zusammengefasst – z. B. können Schüler und Lehrer als Klasse „Mensch“ zusammengefasst werden.

Wichtig sind dabei die relevanten Attribute, Methoden zur Kommunikation über Schnittstellen und die Nutzung von Ähnlichkeiten über Vererbung.

Eine Klasse wird als Rechteck mit drei durch Linien getrennten Bereichen dargestellt: Klassenname, Attribute und Methoden.

- **Sichtbarkeit:** `-` für privat, `+` für öffentlich.

Beispiele: `- modell : String` und `+ beschleunigen() : void`

- **Multiplizität:** Zahlenverhältnis zwischen Objekten, z. B. `1`, `0..5`, `0..*`.

Beziehung	Bedeutung
Assoziation	Allgemeine Beziehung zwischen zwei Klassen.
Aggregation	Ganzes-Teile-Beziehung; die Teile existieren eigenständig weiter (leere Raute).
Komposition	Starke Aggregation; die Teile „sterben“ mit dem Ganzen (gefüllte Raute).
Generalisierung	Vererbung zwischen Basisklasse und abgeleiteter Klasse (leeres Dreieck).

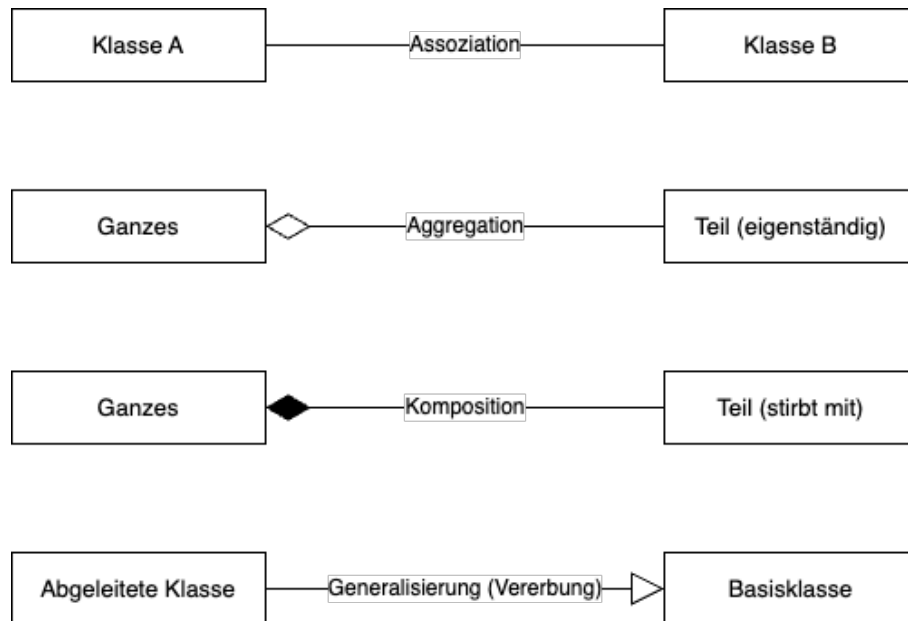


Abbildung 15. Notation der UML-Beziehungen

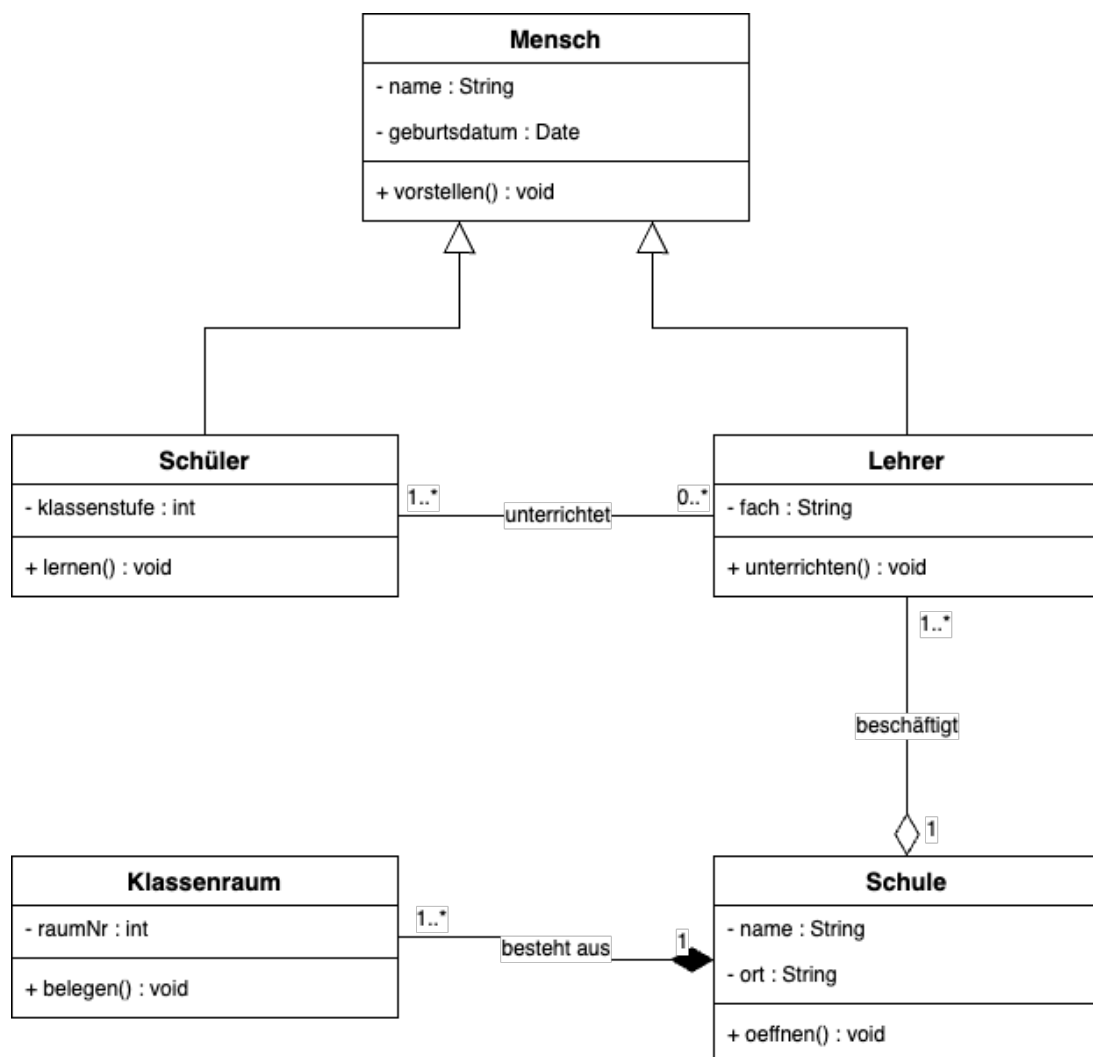


Abbildung 16. Beispiel UML-Klassendiagramm: Schule

Das Beispiel zeigt alle vier Beziehungsarten: *Schüler* und *Lehrer* erben von der Basis-klasse *Mensch* (Generalisierung). Die *Schule* besteht aus *Klassenräumen* (Komposition – ohne Schule kein Klassenraum) und beschäftigt *Lehrer* (Aggregation – ein Lehrer existiert auch ohne die Schule weiter). Zwischen *Lehrer* und *Schüler* besteht die Assoziation „unterrichtet“ mit den Multiplizitäten `1..*` und `0..*`.

9 Normalisierung

Was ist Normalisierung? Das schrittweise Aufteilen einer Relation (Tabelle) in mehrere kleinere Relationen, bis diese den Normalisierungsregeln entsprechen. Ziel ist ein redundanzarmes, konsistentes Schema.

9.1 Anomalien und Redundanz

Unnormalisierte Tabellen speichern Informationen oft mehrfach (Redundanz). Das führt zu drei Arten von Anomalien:

- **Einfügeanomalie** Ein neuer Fakt kann nicht gespeichert werden, ohne einen anderen, eigentlich unabhängigen Fakt mit anzugeben (z. B. kann ein neuer Artikel erst gespeichert werden, wenn es auch schon eine Bestellung für ihn gibt).
- **Änderungsanomalie** Ändert sich ein Fakt, muss er an mehreren Stellen gleichzeitig geändert werden, sonst entstehen Widersprüche (z. B. Adressänderung eines Kunden, die in jeder Bestellzeile einzeln gepflegt wird).
- **Löschanomalie** Beim Löschen eines Datensatzes gehen ungewollt weitere, eigentlich unabhängige Informationen verloren (z. B. löscht man die letzte Bestellung eines Kunden, verschwinden auch dessen Stammdaten).

9.2 Funktionale Abhängigkeit

Ein Attribut B ist *funktional abhängig* von einem Attribut A (geschrieben $A \rightarrow B$), wenn zu jedem Wert von A immer genau ein Wert von B gehört. Der Primärschlüssel einer Relation bestimmt dabei funktional alle übrigen (Nichtschlüssel-)Attribute. Zwei Sonderfälle sind für die Normalformen entscheidend:

- **Teilabhängigkeit (partielle Abhängigkeit)** Ein Nichtschlüsselattribut hängt nur von einem Teil eines zusammengesetzten Schlüssels ab, nicht vom gesamten Schlüssel.
- **Transitive Abhängigkeit** Ein Nichtschlüsselattribut hängt nicht direkt vom Schlüssel ab, sondern über ein anderes Nichtschlüsselattribut ($A \rightarrow C \rightarrow B$).

Als durchgängiges Beispiel dient im Folgenden eine Bestellverwaltung, die schrittweise über alle drei Normalformen normalisiert wird.

9.3 Erste Normalform (1NF)

Eine Relation ist in der 1NF, wenn jeder Attributwert *atomar* ist – es gibt also keine Listen, Mehrfachwerte oder zusammengesetzten Werte in einer Zelle.

Vorher (verletzt 1NF – die Spalte Artikel enthält mehrere Werte):

BestellNr	Kunde	Artikel
1001	Meier	Tastatur, Maus
1002	Schmidt	Monitor

Nachher (1NF – jede Artikelposition steht in einer eigenen Zeile):

BestellNr	Kunde	Artikel
1001	Meier	Tastatur
1001	Meier	Maus
1002	Schmidt	Monitor

9.4 Zweite Normalform (2NF)

Eine Relation ist in der 2NF, wenn sie in 1NF ist *und* jedes Nichtschlüsselattribut voll funktional vom *gesamten* (zusammengesetzten) Primärschlüssel abhängt – es darf also keine Teilabhängigkeit geben. Bei einem einfachen (nicht zusammengesetzten) Primärschlüssel ist die 2NF automatisch erfüllt.

Vorher (Schlüssel = BestellNr + ArtikelNr; verletzt 2NF – Kunde hängt nur von BestellNr, ArtikelBezeichnung nur von ArtikelNr ab):

BestellNr	ArtikelNr	ArtikelBezeichnung	Menge	Kunde
1001	A1	Tastatur	1	Meier
1001	A2	Maus	1	Meier
1002	A3	Monitor	2	Schmidt

Nachher (2NF – aufgeteilt in drei Relationen):

Bestellung		
BestellNr	Kunde	
1001	Meier	
1002	Schmidt	
Artikel		
ArtikelNr	ArtikelBezeichnung	
A1	Tastatur	
A2	Maus	
A3	Monitor	
Bestellposition		
BestellNr	ArtikelNr	Menge
1001	A1	1
1001	A2	1
1002	A3	2

9.5 Dritte Normalform (3NF)

Eine Relation ist in der 3NF, wenn sie in 2NF ist *und* kein Nichtschlüsselattribut transitiv vom Primärschlüssel abhängt. Jedes Nichtschlüsselattribut muss also direkt vom Schlüssel abhängen, nicht über ein anderes Nichtschlüsselattribut.

Vorher (Schlüssel = KundenNr; verletzt 3NF – Ort hängt über PLZ transitiv von KundenNr ab):

KundenNr	Name	PLZ	Ort
K1	Meier	10115	Berlin
K2	Schmidt	80331	München

Nachher (3NF – Ort ist über die PLZ ausgelagert):

Kunde		
KundenNr	Name	PLZ
K1	Meier	10115
K2	Schmidt	80331

Postleitzahl	
PLZ	Ort
10115	Berlin
80331	München

9.6 Zusammenfassung

NF	Regel	Beispiel-Verstoß
1NF	Jeder Attributwert ist atomar – keine Listen, Mehrfach- oder zusammengesetzten Werte.	„Tastatur, Maus“ in einer Zelle der Spalte Artikel.
2NF	1NF + jedes Nichtschlüsselattribut hängt voll vom gesamten (zusammengesetzten) Primärschlüssel ab – keine Teilabhängigkeit.	Kunde hängt nur von BestellNr ab, nicht von BestellNr + ArtikelNr.
3NF	2NF + kein Nichtschlüsselattribut hängt transitiv vom Schlüssel ab.	PLZ → Ort: Ort hängt über die PLZ am Schlüssel.

Hinweis: Gibt es keinen zusammengesetzten Primärschlüssel, ist eine Relation in 1NF automatisch auch in 2NF.

10 SQL

SQL steht für *Structured Query Language* und ist die Standardsprache, um relationale Datenbanken zu definieren, zu füllen, abzufragen und zu verwalten. SQL ist deklarativ: Man beschreibt, *was* man haben möchte, und nicht, *wie* die Datenbank das Ergebnis beschaffen soll. Die Sprache wird in vier Teilsprachen unterteilt; die Transaktionssteuerung (TCL) wird oft als fünfte Teilsprache ergänzt.

Teilsprache	Aufgabe	Wichtige Befehle
DDL – Data Definition Language	Schema der Datenbank definieren und verändern	CREATE , ALTER , DROP , TRUNCATE
DML – Data Manipulation Language	Daten in bestehenden Tabellen einfügen, ändern und löschen	INSERT , UPDATE , DELETE , REPLACE
DQL – Data Query Language	Daten aus einer oder mehreren Tabellen lesen	SELECT
DCL – Data Control Language	Zugriff und Rechte für Nutzer steuern	GRANT , REVOKE
TCL – Transaction Control Language	Transaktionen steuern	COMMIT , ROLLBACK

10.1 DDL – Data Definition Language

Die DDL umfasst alle Befehle, die die *Struktur* einer Datenbank definieren und verändern. Es geht nicht um die Daten selbst, sondern um das Schema. Fast jedes Datenbankobjekt kann mit `CREATE` erzeugt, mit `ALTER` verändert, mit `RENAME` / `CHANGE` umbenannt und mit `DROP` gelöscht werden.

10.1.1 Datenbank und Tabelle anlegen

Mit `IF NOT EXISTS` wird eine Exception abgefangen, falls die Datenbank bereits existiert.

```
CREATE DATABASE IF NOT EXISTS kurs_db;

CREATE TABLE Personal (
    PersonalNummer INT UNSIGNED,           -- kein NOT NULL noetig (ist PK)
    Vorname        VARCHAR(60) NOT NULL,
    Nachname       VARCHAR(60) NOT NULL,
    GeburtsDatum   DATE NOT NULL,
    AbteilungsID   SMALLINT UNSIGNED NOT NULL,
    PRIMARY KEY (PersonalNummer),
    FOREIGN KEY (AbteilungsID) REFERENCES Abteilung(AbteilungsID)
```

```
);
```

10.1.2 Datentypen

Art	Typen
Text	<code>CHAR(n)</code> feste Länge · <code>VARCHAR(n)</code> variable Länge, maximal n Zeichen · <code>TEXT</code> / <code>BLOB</code> sehr große Inhalte
Ganze Zahlen	<code>INT</code> / <code>INTEGER</code> , <code>SMALLINT</code> , <code>TINYINT</code>
Gleitkommazahlen	<code>FLOAT</code> , <code>DOUBLE</code> – z. B. <code>FLOAT(8,2)</code> mit 2 Nachkommastellen
Logisch	<code>BOOL</code> (intern ein <code>TINYINT</code>)
Zeiten	<code>DATE</code> · <code>TIME</code> · <code>DATETIME</code> · <code>TIMESTAMP</code> (setzt bei <code>INSERT</code> / <code>UPDATE</code> automatisch Datum und Uhrzeit) · <code>YEAR</code>

10.1.3 Spaltenattribute und Constraints

Schlüsselwort	Bedeutung
<code>PRIMARY KEY</code>	Primärschlüssel: eindeutig, nie <code>NULL</code> , identifiziert jeden Datensatz. Kann auch aus mehreren Spalten bestehen.
<code>FOREIGN KEY</code>	Fremdschlüssel: verweist auf den Primärschlüssel einer anderen Tabelle.
<code>NOT NULL</code>	Die Spalte darf keinen <code>NULL</code> -Wert enthalten.
<code>UNIQUE</code>	Alle Werte der Spalte müssen eindeutig sein.
<code>DEFAULT</code>	Standardwert, falls beim Einfügen kein Wert angegeben wird.
<code>AUTO_INCREMENT</code>	Nummeriert Datensätze automatisch aufsteigend ab 1; nur für ganzzahlige, eindeutige Felder (<code>PRIMARY KEY</code> oder <code>UNIQUE</code>).
<code>UNSIGNED</code>	Nur positive Zahlen erlaubt (kein Vorzeichen).

Stolperfalle

In SQL kann man eine Tabelle auch ohne `PRIMARY KEY` erstellen – das sollte man aber nicht tun und den Schlüssel bereits bei der Modellierung festlegen.

10.1.4 Struktur ändern mit ALTER TABLE

`ALTER TABLE` verändert die Struktur einer bestehenden Tabelle, ohne die Daten zu löschen.

```
ALTER TABLE Personal ADD Gender CHAR AFTER GeburtsDatum; -- Spalte
                        hinzufuegen
ALTER TABLE Personal MODIFY GeburtsDatum DATETIME;      -- Datentyp
                        aendern
```

```

ALTER TABLE Personal CHANGE GeburtsDatum Geburtsdatum DATE; -- Spalte
                        umbenennen
ALTER TABLE Personal DROP Gender;                                -- Spalte
                        loeschen
ALTER TABLE Personal RENAME TO Mitarbeiter;                      -- Tabelle
                        umbenennen

-- Foreign Key nachtraeglich setzen
ALTER TABLE Personal
    ADD FOREIGN KEY (AbteilungsID) REFERENCES Abteilung(AbteilungsID);

```

10.1.5 Referenzielle Integrität

Die Tabelle mit dem Primärschlüssel ist der *Parent* (Master), die Tabelle mit dem Fremdschlüssel das *Child*. Die referenzielle Integrität verhindert verwaiste Daten – etwa eine Bestellung ohne existierenden Kunden. Mit `ON DELETE` und `ON UPDATE` legt man fest, was mit den Child-Datensätzen passiert:

Option	Verhalten
<code>RESTRICT</code> / <code>NO ACTION</code>	Löschen bzw. Ändern des Parent-Datensatzes wird verhindert.
<code>CASCADE</code>	Child-Datensätze werden mitgelöscht bzw. mitgeändert.
<code>SET NULL</code>	Der Fremdschlüssel im Child wird auf <code>NULL</code> gesetzt.

10.1.6 Löschen: DROP, TRUNCATE, DELETE

Mit `DROP` wird eine Tabelle oder Datenbank komplett entfernt, mit `TRUNCATE` werden nur alle Zeilen gelöscht – die Tabelle selbst bleibt bestehen.

```

DROP DATABASE kurs_db;      -- Datenbank komplett entfernen
DROP TABLE Personal;      -- Tabelle komplett entfernen
TRUNCATE TABLE Personal;  -- alle Zeilen loeschen, Struktur bleibt

```

Befehl	Löscht Daten?	Löscht Struktur?
<code>DROP TABLE</code>	Ja	Ja – Tabelle ist weg
<code>TRUNCATE TABLE</code>	Ja	Nein – Tabelle bleibt
<code>DELETE FROM</code>	Ja (mit <code>WHERE</code> filterbar)	Nein – Tabelle bleibt

10.2 DML – Data Manipulation Language

Die DML umfasst alle Befehle, die Daten *innerhalb* bestehender Tabellen verändern: einfügen (`INSERT`), ändern (`UPDATE`), ersetzen (`REPLACE`) und löschen (`DELETE`).

10.2.1 INSERT

```
-- Mit Spaltenliste (empfohlen), mehrere Datensätze auf einmal
INSERT INTO Leistung (LNr, Bezeichnung, Kosten, AID) VALUES
    (800, 'FamilienRecht', 67.00, NULL),
    (801, 'ArbeitsRecht', 700.00, NULL);

-- Ohne Spaltenliste: nur wenn ALLE Spalten in richtiger Reihenfolge
  angegeben
INSERT INTO Leistung VALUES (804, 'Sonstiges', 99.00, NULL);
```

Hinweis zur Spaltenliste

Die runden Klammern nach dem Tabellennamen (die Spaltenliste) kann man weglassen, wenn man alle Spalten in der korrekten Reihenfolge angibt. Die Klammern bei `VALUES (...)` bleiben immer.

10.2.2 UPDATE, REPLACE und DELETE

```
-- Eine oder mehrere Spalten ändern
UPDATE Leistung
SET AID = 2, Kosten = 99.00
WHERE LNr = 17;

-- Bestehenden Datensatz komplett ersetzen
REPLACE kurs_db.Abteilung VALUES (4, 'Rechnungswesen');

-- Datensätze mit Bedingung löschen
DELETE FROM Leistung
WHERE LNr = 800;
```

Achtung

`UPDATE` und `DELETE` ohne `WHERE` verändern bzw. löschen *alle* Zeilen der Tabelle. Vor dem Ausführen immer die `WHERE`-Bedingung prüfen.

10.3 DQL – Data Query Language

`SELECT` liest Daten aus einer oder mehreren Tabellen. Das Ergebnis ist immer wieder eine Tabelle (Mengeeigenschaft).

10.3.1 Aufbau eines SELECT-Statements

Die Schreibreihenfolge ist fest vorgegeben:

```

SELECT   Spalten / Aggregatfunktionen
FROM     Tabellenname
WHERE     Bedingung           -- Zeilen filtern (VOR der Gruppierung)
GROUP BY Spalte              -- Gruppen bilden
HAVING    Bedingung           -- Gruppen filtern (NACH der Gruppierung)
ORDER BY  Spalte ASC / DESC -- Sortieren
LIMIT     Anzahl;            -- Anzahl begrenzen

```

Die Schreibreihenfolge und die interne Ausführungsreihenfolge sind unterschiedlich:

Schritt	Befehl
1	FROM
2	WHERE
3	GROUP BY
4	HAVING
5	SELECT
6	ORDER BY
7	LIMIT

Begriff	Bedeutung
Horizontal	bestimmte <i>Zeilen</i> filtern (WHERE)
Vertikal	bestimmte <i>Spalten</i> auswählen (SELECT s1, s2)

```

SELECT * FROM MA;           -- * = alle Spalten
SELECT VName, Gehalt FROM MA; -- nur bestimmte Spalten (vertikal)
SELECT name AS Kundenname FROM kunde; -- AS = Spalten-Alias

```

10.3.2 WHERE – Zeilen filtern

Mit **WHERE** definiert man Einschränkungen (horizontales Filtern).

Operator	Bedeutung
> / >=	größer / größer gleich
< / <=	kleiner / kleiner gleich
=	Gleichheit
!= oder <>	ungleich
AND	beide Bedingungen müssen wahr sein
OR	mindestens eine Bedingung muss wahr sein
NOT	kehrt die Bedingung um
XOR	genau eine Bedingung darf wahr sein

Klammern steuern die Reihenfolge der Auswertung:

```

SELECT id_kunde, name FROM kunde
WHERE (ort = 'Muenster' OR YEAR(geb_datum) > 1960) AND id_kunde > 2;

-- BETWEEN: Bereich (inklusive Grenzen)
WHERE Gehalt BETWEEN 2000 AND 4000
-- entspricht: WHERE Gehalt >= 2000 AND Gehalt <= 4000

-- LIKE: Mustererkennung
WHERE VName LIKE 'M%' -- beginnt mit M (% = beliebig viele Zeichen)
WHERE VName LIKE 'M_' -- M + genau 1 Zeichen (_ = genau ein Zeichen)

-- IN: ersetzt eine Kette an OR-Bedingungen
WHERE AID IN (1, 2, 3)
-- entspricht: WHERE AID = 1 OR AID = 2 OR AID = 3

```

10.3.3 Funktionen

Aggregatfunktionen fassen mehrere Zeilen zu einem einzelnen Wert zusammen. Sie arbeiten immer auf einer ganzen Spalte oder Gruppe. Daneben gibt es Zeichenketten- und Zeitfunktionen.

Funktion	Bedeutung
COUNT(*)	Anzahl der Zeilen
SUM(spalte)	Summe aller Werte
AVG(spalte)	Durchschnitt aller Werte (NULL wird ignoriert)
MIN(spalte) / MAX(spalte)	kleinster / größter Wert
LENGTH(s)	Anzahl der Zeichen: LENGTH('frie') → 4
LEFT(s, n) / RIGHT(s, n)	die ersten / letzten n Zeichen einer Zeichenkette
CURDATE() / CURTIME()	aktuelles Datum / aktuelle Uhrzeit
CURRENT_TIMESTAMP	aktuelles Datum samt Uhrzeit
YEAR(datum)	Jahr eines Datums

```

SELECT
    COUNT(*) AS AnzahlMitarbeiter, -- AS vergibt einen Alias
    AVG(Gehalt) AS Durchschnittsgehalt,
    MIN(Gehalt) AS MinGehalt,
    MAX(Gehalt) AS MaxGehalt,
    SUM(Gehalt) AS Gesamtlohn
FROM MA;

```

10.3.4 GROUP BY und HAVING

GROUP BY fasst Zeilen mit dem gleichen Wert in einer Spalte zu einer Gruppe zusammen. Danach kann man pro Gruppe eine Aggregatfunktion anwenden. WHERE filtert Zeilen

vor der Gruppierung, **HAVING** filtert Gruppen *nach* der Gruppierung.

```
-- Ohne GROUP BY: ein Wert ueber alle Zeilen
SELECT AVG(Gehalt) FROM MA;

-- Mit GROUP BY: ein Wert pro Abteilung
SELECT AID, AVG(Gehalt) AS Durchschnittsgehalt
FROM MA
GROUP BY AID;

SELECT AID, COUNT(*) AS Anzahl
FROM MA
WHERE Gehalt > 2000      -- filtert Zeilen BEVOR Gruppen gebildet werden
GROUP BY AID
HAVING COUNT(*) > 2;    -- filtert Gruppen NACHDEM sie gebildet wurden
```

Wichtige Regel

Jede Spalte im **SELECT**, die keine Aggregatfunktion ist, muss auch im **GROUP BY** stehen – sonst gibt es einen Fehler.

10.3.5 ORDER BY und LIMIT

```
SELECT VName, Gehalt FROM MA
ORDER BY Gehalt DESC;  -- DESC = absteigend, ASC = aufsteigend (Standard)

SELECT * FROM MA LIMIT 5;      -- MySQL
SELECT * FROM MA FETCH FIRST 5 ROWS ONLY;  -- Standard-SQL
```

10.3.6 Joins

Joins verknüpfen Daten aus mehreren Tabellen über eine Beziehung – meist Primärschlüssel (Parent) = Fremdschlüssel (Child).

Join-Typ	Was wird ausgegeben?
INNER JOIN	Nur Datensätze mit Treffer in <i>beiden</i> Tabellen (Schnittmenge).
LEFT JOIN	Alle Datensätze der linken Tabelle – auch ohne Treffer rechts.
RIGHT JOIN	Alle Datensätze der rechten Tabelle – auch ohne Treffer links.
CROSS JOIN	Kartesisches Produkt: jede Zeile wird mit jeder kombiniert.

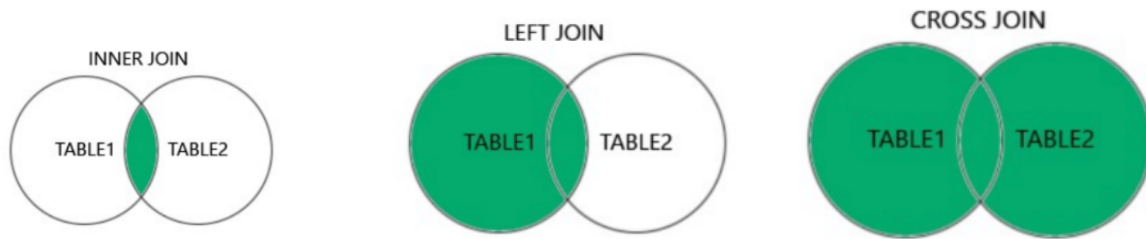


Abbildung 17. Join-Typen als Schnittmengendiagramm: Inner Join, Left Join und Cross Join

```
SELECT bestelldatum, name FROM kunde
INNER JOIN bestellung ON kunde.id_kunde = bestellung.id_kunde;
```

Merkhilfe: Schnittmengendiagramm

INNER = Überschneidung der beiden Kreise, **LEFT / RIGHT** = der ganze linke bzw. rechte Kreis. Vorsicht beim **CROSS JOIN**: Das voll eingefärbte Diagramm steht hier für „alle Kombinationen“ – ein Cross Join hat keine **ON**-Bedingung und liefert bei m und n Zeilen $m \times n$ Ergebniszeilen. Die komplette Vereinigungsmenge beider Kreise wäre streng genommen ein **FULL OUTER JOIN**.

10.3.7 UNION

Während ein Join Tabellen *horizontal* verknüpft (Spalten nebeneinander), hängt **UNION** die Ergebnisse zweier **SELECT**-Abfragen *vertikal* aneinander (Zeilen untereinander). Voraussetzung: Beide Abfragen müssen die gleiche Spaltenanzahl mit kompatiblen Datentypen liefern. **UNION** entfernt dabei Duplikate, **UNION ALL** behält sie.

```
SELECT name, ort FROM kunde
UNION                                -- Duplikate werden entfernt
SELECT name, ort FROM lieferant;

SELECT name, ort FROM kunde
UNION ALL                            -- Duplikate bleiben erhalten
SELECT name, ort FROM lieferant;
```

FULL OUTER JOIN in MySQL

MySQL kennt keinen **FULL OUTER JOIN** – man baut ihn mit **UNION** nach: das Ergebnis eines **LEFT JOIN** per **UNION** mit dem Ergebnis des passenden **RIGHT JOIN** vereinigen.

10.3.8 Unterabfragen (Subselects)

Wie in der Mathematik bei $f(g(x))$: Das innere Statement wird zuerst ausgeführt, sein Ergebnis nutzt die äußere Abfrage weiter. Das Ergebnis einer Unterabfrage ist immer eine Tabelle. Vorteil: Ein Teilproblem wird getrennt gelöst – oft eine übersichtlichere Alternative zum Join.

```
-- Unterabfrage im FROM (braucht immer einen Alias)
SELECT * FROM
    (SELECT VName FROM MA) AS temp; -- AS temp ist Pflicht

-- Subselect im WHERE: zuerst MAX berechnen, dann damit vergleichen
SELECT VName, Gehalt FROM MA
WHERE Gehalt = (SELECT MAX(Gehalt) FROM MA);
```

10.3.9 Views (Sichten)

Eine View ist eine gespeicherte `SELECT`-Abfrage, die wie eine virtuelle Tabelle genutzt wird. Im Unterschied zu einer Tabelle speichert eine View keine eigenen Daten, sondern nur die Abfrage – die Daten liegen weiter in den Basistabellen.

Vorteile

- Komplexe Abfragen unter einem Namen speichern
- Einheitliche Darstellung für alle Nutzer
- Eingeschränkter, datensparsamer Zugriff

Nachteile

- Keine eigenen Daten – Performance hängt an den Basistabellen

10.4 DCL – Data Control Language

Die DCL steuert Zugriff und Rechte auf die Datenbank. Kernfrage des Datenschutzes: *Wer darf welche Daten zu welchem Zweck sehen oder ändern?*

Grundsatz

Jeder User bekommt nur die Rechte, die er wirklich benötigt – kein pauschales

`ALL`.

`GRANT` vergibt Rechte, `REVOKE` entzieht sie. Rechte sind u. a. `ALL`, `SELECT`, `INSERT`, `UPDATE`, `DELETE`, `DROP`. Es gibt drei Ebenen: global (`*.*`), Datenbank (`db.*`) und Tabelle (`db.tabelle`).

```
-- Nutzer anlegen und loeschen
CREATE USER 'test'@'localhost' IDENTIFIED BY 'pw123';
DROP USER 'test'@'localhost';
```

```
-- Rechte vergeben und entziehen
GRANT SELECT ON kurs_db.kunde TO 'test'@'localhost';
GRANT ALL      ON kurs_db.*      TO 'test'@'localhost';
REVOKE ALL     ON kurs_db.mitarbeiter FROM 'test'@'localhost';
```

10.5 TCL – Transaktionen

Eine *Transaktion* ist eine logisch zusammenhängende Gruppe von SQL-Befehlen, die vollständig oder gar nicht ausgeführt wird – z. B. eine Überweisung: abbuchen *und* gutschreiben.

```
START TRANSACTION;
UPDATE konto SET saldo = saldo - 100 WHERE id_konto = 1;
UPDATE konto SET saldo = saldo + 100 WHERE id_konto = 2;
COMMIT;          -- dauerhaft speichern
-- ROLLBACK; -- alternativ: alle Änderungen verwerfen
```

Transaktionen arbeiten zuverlässig nach dem **ACID**-Prinzip:

Eigenschaft	Bedeutung
A tomicity	Alles oder nichts – eine Transaktion wird nie teilweise ausgeführt.
C onsistency	Die Datenbank ist vor und nach der Transaktion konsistent.
I solation	Gleichzeitige Transaktionen beeinflussen sich nicht.
D urability	Nach dem COMMIT sind die Änderungen dauerhaft gespeichert.

10.5.1 Locking

Beim *Locking* werden Datenbankobjekte (Zelle, Zeile, Tabelle oder ganze Datenbank) temporär gesperrt; die Freigabe erfolgt meist nach dem Transaktionsende.

- **Shared Lock (Read Lock):** Mehrere Transaktionen dürfen gleichzeitig lesen, keine darf schreiben.
- **Exclusive Lock (Write Lock):** Nur eine Transaktion darf lesen und schreiben; alle anderen müssen warten.

Gegenseitiges Sperren kann zu einem *Deadlock* führen.

10.5.2 Isolationsebenen

Isolationsebenen legen fest, wie stark gleichzeitige Transaktionen voneinander getrennt werden (von der schwächsten zur stärksten): **READ UNCOMMITTED** → **READ COMMITTED** → **REPEATABLE READ** → **SERIALIZABLE**. Höhere Isolation bedeutet weniger Nebeneffekte, aber geringere Parallelität.

10.5.3 Events

Ein *Event* ist eine automatisch geplante SQL-Anweisung, die zu einer bestimmten Zeit oder regelmäßig ausgeführt wird – z. B. um alte Daten zu löschen, Auswertungen vorzubereiten oder Logs zu bereinigen.

10.6 Trigger, Funktionen und Prozeduren

10.6.1 Trigger

Ein *Trigger* fängt ein Ereignis ab (`INSERT` , `UPDATE` oder `DELETE` an einer Tabelle) und führt automatisch einen Anweisungsblock aus („feuern“). Er wird nie vom Benutzer aufgerufen, sondern vom DBMS – entweder `BEFORE` oder `AFTER` der Änderung. Typische Einsatzzwecke: Werte automatisch setzen, Plausibilität prüfen, Änderungen protokollieren und referenzielle Integrität überwachen.

10.6.2 Funktionen

Eine benutzerdefinierte Funktion ist eine *Skalarfunktion*: Sie gibt genau einen Wert zurück. Der Rückgabebetyp wird mit `RETURNS` deklariert, der Wert mit `RETURN` zurückgegeben.

Nach `DETERMINISTIC` folgt der eigentliche Funktionskörper.

```
CREATE FUNCTION multipliziere(param1 FLOAT(8,2), param2 FLOAT(8,2))
RETURNS FLOAT(8,2)
READS SQL DATA
DETERMINISTIC
RETURN param1 * param2;

SELECT multipliziere(Kosten, 1.19) FROM Leistung;    -- Aufruf
```

10.6.3 Prozeduren (Stored Procedures)

Eine *Stored Procedure* ist ein benannter, in der Datenbank gespeicherter Anweisungsblock – mit oder ohne Parameter, aufgerufen per `CALL`. Nutzen: Businesslogik in die Datenbank auslagern, Wiederverwendbarkeit von mehreren Rechnern aus und Parameterübergabe.

11 Glossar

ACID

Atomicity, Consistency, Isolation, Durability – die vier Eigenschaften zuverlässiger Transaktionen.

Aggregatfunktion

Funktion, die mehrere Zeilen zu einem einzelnen Wert zusammenfasst: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`.

Anomalie

Fehler, der durch Redundanz in unnormalisierten Tabellen entsteht (Einfüge-, Änderungs- und Löschanomalie).

ANSI-3-Ebenen-Modell

Architektur, die eine Datenbank in eine interne, eine konzeptionelle und eine externe Ebene trennt (auch *SPARC* genannt).

Attribut

Eigenschaft eines Objekts bzw. Spalte einer Relation.

CAP-Theorem

Konsistenz, Verfügbarkeit und Partitionstoleranz sind in einem verteilten System nie gleichzeitig vollständig erreichbar – nur zwei davon.

Caching

Clients halten eine eigene, lokale Kopie der Daten (oder von Teilen davon) – schneller Zugriff, aber Konsistenzfragen.

Chen-Notation

Standardnotation für ER-Diagramme: Rechteck = Objekttyp, Ellipse = Eigenschaft, Raute = Beziehungstyp.

Constraint

Regel auf einer Spalte oder Tabelle, z. B. `NOT NULL` oder `UNIQUE`.

CQRS

Command Query Responsibility Segregation – Architekturmuster, das Schreiben (Commands, Write-Modell) und Lesen (Queries, Read-Modell) strikt trennt; Synchronisation über Events.

Data Mart

Abteilungs- oder funktionsbezogene Teilmenge eines Data Warehouse.

Data Mining

Automatisches Entdecken von Modellen und Mustern in großen Datenbeständen.

Data Warehouse

Sehr große, von den operativen Systemen unabhängige Datenbank mit verdichteten Daten für Auswertungen.

Datenwürfel (Cube)

Mehrdimensionale Sicht auf Kennzahlen: Jede Zelle enthält eine Kennzahl am Schnittpunkt der Dimensionen.

DBMS

Datenbank-Management-System – verwaltet die Datenbank und ist die Schnittstelle zu ihr (z. B. MySQL, PostgreSQL).

DCL

Data Control Language – steuert Zugriff und Rechte (`GRANT` , `REVOKE`).

DDL

Data Definition Language – definiert und verändert das Schema (`CREATE` , `ALTER` , `DROP`).

Deadlock

Gegenseitiges Sperren zweier Transaktionen, die aufeinander warten.

Dimension

Blickwinkel, aus dem eine Kennzahl (Fakt) betrachtet wird – z. B. Zeit, Produkt, Kunde oder Region; besitzt Hierarchien (Tag → Monat → Jahr).

DML

Data Manipulation Language – verändert Daten in bestehenden Tabellen (`INSERT` , `UPDATE` , `DELETE`).

DQL

Data Query Language – fragt Daten ab (`SELECT`).

DTO

Data Transfer Object – Objekt, das Daten zwischen Anwendung und API transportiert; Teil der externen Ebene.

Entität

Konkretes Objekt der realen Welt, das in der Datenbank modelliert wird.

ERM

Entity-Relationship-Modell – konzeptioneller Entwurf einer Datenbank mit Entitäten, Attributen und Beziehungen.

Event

Automatisch geplante SQL-Anweisung, die zu einer bestimmten Zeit oder regelmäßig ausgeführt wird.

Foreign Key

Fremdschlüssel – verweist auf den Primary Key einer anderen Tabelle und verknüpft so die Daten.

Funktionale Abhängigkeit

$A \rightarrow B$: Zu jedem Wert von A gehört genau ein Wert von B .

Isolationsebene

Legt fest, wie stark gleichzeitige Transaktionen getrennt werden: `READ UNCOMMITTED`

bis `SERIALIZABLE` .

Join

Verknüpft Daten aus mehreren Tabellen über Primär- und Fremdschlüssel (`INNER` , `LEFT` , `RIGHT` , `CROSS`).

Kardinalität

Gibt an, wie viele Objekte eines Typs an einer Beziehung teilnehmen können (1:1, 1:n, n:m).

Locking

Temporäres Sperren von Datenbankobjekten – *Shared Lock* zum Lesen, *Exclusive Lock* zum Schreiben.

Managementunterstützungssystem

System zur Extraktion von Wissen aus großen Datenmengen für Berichte über vergangene Daten – Basis: Data Warehouse, OLAP und Data Mining.

Map/Reduce

Programmiermodell zur Parallelisierung nach dem Prinzip *Divide and Conquer*.

Multiplizität

Zahlenverhältnis zwischen Objekten im UML-Klassendiagramm, z. B. `1` , `0..5` , `0..*` .

Normalisierung

Schrittweises Aufteilen von Relationen (1NF–3NF), um Redundanz und Anomalien zu vermeiden.

NoSQL

Not Only SQL – Alternative zu relationalen DBMS (Key-Value, Document Store, Column Store, Graphdatenbank).

OLAP

Online Analytical Processing – flexible, mehrdimensionale Datenanalyse nach dem Top-down-Ansatz.

OLTP

Online Transaction Processing – Verarbeitung von Transaktionen in der Anwendungsdatenbank; schreiboptimiert und normalisiert.

ORM

Object-Relational Mapper – bildet Objekte einer Anwendung auf relationale Tabellen ab (z. B. Entity Framework Core).

Primary Key

Primärschlüssel – identifiziert einen Datensatz eindeutig, ist nie `NULL` und kann aus mehreren Spalten bestehen.

Prozedur

Benannter, in der Datenbank gespeicherter Anweisungsblock; Aufruf per `CALL` .

RDM

Relationales Datenbankmodell – Daten liegen in Tabellen (Relationen) mit Primär- und Fremdschlüsseln, in der dritten Normalform.

Redundanz

Mehrfaches Speichern derselben Information.

Referenzielle Integrität

Verhindert verwaiste Datensätze: Ein Fremdschlüssel muss auf einen existierenden Primärschlüssel zeigen.

Relation

Tabelle im relationalen Modell; eine Zeile ist ein *Tupel*, ein Datensatz ein *Record*.

Replikation

Dieselben Daten liegen auf mehreren Nodes – entweder Primary/Secondary oder Peer-to-Peer.

Sharding

Aufteilen des Datenbestands: Jeder Node hält nur einen Teil der Daten.

Skalarfunktion

Benutzerdefinierte Funktion, die genau einen Wert zurückgibt (`RETURNS` deklariert den Typ, `RETURN` den Wert).

Star-Schema

Datenmodell des Data Warehouse: Faktentabelle mit Kennzahlen und Fremdschlüsseln in der Mitte, sternförmig umgeben von bewusst denormalisierten Dimensionstabellen.

Subselect

Verschachtelte Abfrage; das innere `SELECT` wird zuerst ausgeführt, sein Ergebnis nutzt die äußere Abfrage.

Teilabhängigkeit

Ein Nichtschlüsselattribut hängt nur von einem Teil eines zusammengesetzten Schlüssels ab – verletzt die 2NF.

Transaktion

Logisch zusammenhängende Gruppe von SQL-Befehlen, die vollständig oder gar nicht ausgeführt wird.

Transitive Abhängigkeit

Ein Nichtschlüsselattribut hängt über ein anderes Nichtschlüsselattribut vom Schlüssel ab ($A \rightarrow C \rightarrow B$) – verletzt die 3NF.

Trigger

Vom DBMS automatisch ausgelöster Anweisungsblock – `BEFORE` oder `AFTER` einem `INSERT`, `UPDATE` oder `DELETE`.

Tupel

Zeile einer Relation; Liste fester Länge aus Attributwerten.

UNION

Vereinigt die Ergebnisse zweier `SELECT`-Abfragen zeilenweise (vertikal); entfernt Duplikate, `UNION ALL` behält sie.

Verteiltes System

Sammlung mehrerer unabhängiger Computer bzw. Prozesse, die nach außen wie ein einziges System wirken.

View

Gespeicherte `SELECT`-Abfrage, die wie eine virtuelle Tabelle genutzt wird – ohne eigene Daten.